

Module -1

BIG DATA ANALYTICS-Introduction to Big Data Analytics

Introduction to Big Data Analytics: Big Data, Scalability and Parallel Processing, Designing Data Architecture, Data Sources, Quality, Pre-Processing and Storing, Data Storage and Analysis.

SLT: Applications of Big Data in the real world.

Textbook 1: Ch1: 1.2 -1.7

What is Data?

Data" is a broad term for raw facts, figures, or values that can be recorded, stored, and processed.

*On its own, data doesn't have much meaning — it's just observations or measurements. When you organize, process, and interpret data, it becomes **information** that people can actually use.*

Everyday examples: a temperature reading (72°F), a name in a contact list, a pixel's color in a photo, a timestamp on a purchase.

A DAY OF DATA

402M

terabytes of data created worldwide, every single day

5.4B

internet users generating data continuously

328M+

TB average daily volume estimated for 2025

Illustrative figures — used to convey scale, not exact statistics.

Data vs. Information vs. Knowledge

Data: "72, 68, 75, 71" (raw numbers)

Information: "The average temperature this week was 71.5°F" (data with context)

Knowledge: "Temperatures like this mean I should plant tomatoes now" (information applied with understanding)

Data-----> Big Data

Need of Big Data

The rise in technology has led to the production and storage of voluminous amounts of data. Earlier megabytes (10^6 B) were used but nowadays petabytes (10^{15} B) are used for processing, analysis, discovering new facts and generating new knowledge.

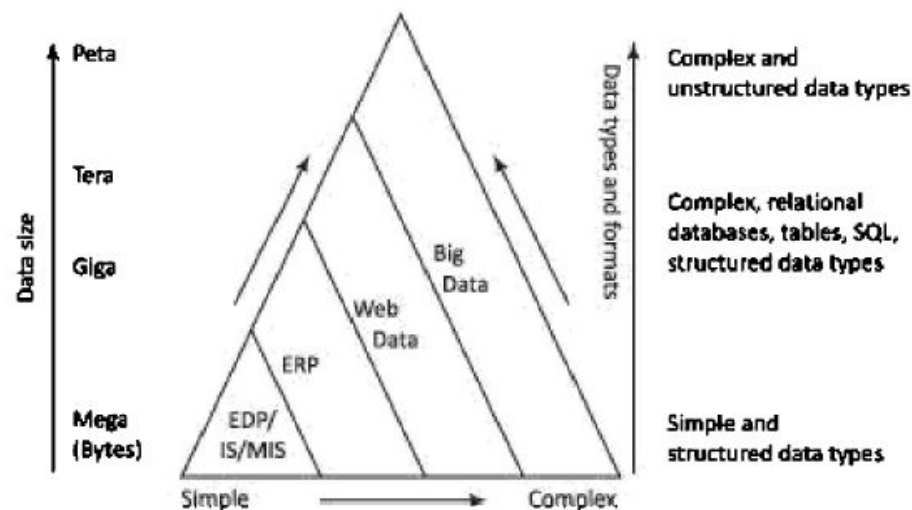


Figure 1.1 Evolution of Big Data and their characteristics

Analytics?

Data: "72, 68, 75, 71" (raw numbers)

Information: "The average temperature this week was 71.5°F" (data with context)

Knowledge: "Temperatures like this mean I should plant tomatoes now" (information applied with understanding)

From Chess Strategy to Big Data Analytics

How studying past games mirrors how organizations learn from data



The 5 V's of Big Data

A simple framework to evaluate whether something is truly a “Big Data” problem



Volume

The sheer amount of data — terabytes to exabytes



Velocity

The speed data is generated and must be processed



Variety

Structured, unstructured & mixed formats



Veracity

Trustworthiness and quality of the data



Value

Whether it yields real, usable insight

Keywords

Application

API

Data Model

Data Repository

Data Stores

Distributed Data Stores

Database

Table

Flat File

CSV

Key-Value Pair

Spreadsheet

Hash-Key Value Pair

Database Administration

Definitions

Data: Data has several definitions, usages can be singular or plural

1. Data is information, usually in the form of facts or statistics that one can analyze or use for further calculations.
2. Data is information that can be stored and used by a computer program
3. Data is information presented in numbers, characters, words, or other form
4. Data is information from series of observations, measurements, or facts

Definitions

Web Data:

Web is large scale integration and presence of data on web servers. Web is a part of the Internet that stores web data in the form of documents and other web resources. URLs enable the access to web data resources.

Examples: Wikipedia, google, and social networks

Classification of Data

- 1. Structured**
- 2. Semi-structured**
- 3. Unstructured**

Classification of Data

- 1. Structured:** Data that is organized in a fixed format, typically in rows and columns, making it easily searchable and analyzable using traditional database tools.

Characteristics:

- *Follows a predefined schema/model*
- Stored in relational databases (RDBMS)
- Easily queried using SQL
- Highly organized and consistent format

Classification of Data

2. Un-Structured: Data that has no predefined format or organization, making it difficult to collect, process, and analyze using traditional methods.

Characteristics:

- No fixed schema or structure
- Cannot be directly stored in traditional relational databases
- Requires advanced tools (NoSQL, Big Data platforms) for storage and analysis

Classification of Data

3. Semi-Structured: Data that does not reside in a relational database but still has some organizational properties (tags, markers) that make it easier to analyze than unstructured data. It doesn't fit neatly into tables but has a self-describing structure.

Characteristics:

- Does not follow a strict schema
- Contains **tags** or **markers** to separate data elements
- Organized but flexible (schema-less or self-describing)
- Easier to analyze than unstructured data, but not as easy as structured data

Classification of Data

4. What is Multi-Structured Data?

Data that coexist within a single system, often coming from multiple sources simultaneously.

Think of it as a **mixed bag of data types**. Instead of data being neatly organized in one format, it's a combination of:

1. **Structured data** – data in neat rows/columns, like an Excel sheet or database table
2. **Semi-structured data** – data that's partly organized but doesn't fit perfectly into a table (like an XML or JSON file)
3. **Unstructured data** – free-form data with no fixed format (like text, images, videos, social media posts)

Big Data Definitions

Big Data is high-*volume*, high-*velocity* and/or high-*variety* information asset that requires new forms of processing for enhanced *decision making*, *insight discovery* and *process optimization* (Gartner¹ 2012).

Industry analyst **Doug Laney** described the '**3Vs**', i.e. volume, variety and/or velocity as the key "data management challenges" for enterprises. Analytics also describe the '**4Vs**', i.e. volume, velocity, variety and veracity.

Big Data Characteristics

Volume	How much data there is the sheer quantity generated by apps/systems
Velocity	How fast data is created and needs to be processed real-time or near real-time
Variety	How many different types/formats of data exist text, images, tables, videos, etc. from different sources, which makes it complex
Veracity	How trustworthy/accurate the data is messy or unreliable data is harder to analyze correctly

Big Data Types

A task team on Big Data classified the types of Big Data (June 2013). Another team from IBM developed a different classification for Big Data types.

Following are the suggested types:

1. ***Social networks and web data***, such as Facebook, Twitter, e-mails, blogs and YouTube.
2. ***Transactions data and Business Processes (BPs) data***, such as credit card transactions, flight bookings, etc. and public agencies data such as medical records, insurance business data etc.
3. ***Customer master data***, such as data for facial recognition and for the name, date of birth, marriage anniversary, gender, location and income category.

4. Machine-generated data, such as machine-to-machine or Internet of Things data, and the data from sensors, trackers, web logs and computer systems log.

Computer generated data is also considered as machine generated data from data store. Usage of programs for processing of data using data repositories, such as database or file, generates data and also machine generated data.

5. Human-generated data such as

- Biometrics data, human-machine interaction data, e-mail records with a mail server and MySQL database of student grades.
- Humans also records their experiences in ways such as writing these in notebooks or diaries, taking photographs or audio and video clips.
- Human-sourced information is now almost entirely digitized and stored everywhere from personal computers to social networks. Such data are loosely structured and often ungoverned.

1. "Amit Kumar, 25, Mysuru, Rating:5"
2. {"name":"Sara Khan","age":30,"city":"Bengaluru","rating":3}
3. "The delivery was late and the packaging was damaged, very disappointed"
4. "Ravi, 22, Chennai, Rating:4"
5. "Great service! Fast delivery. Highly recommend to everyone"
6. {"name":"Divya","age":28,"city":"Mysuru","rating":5}
7. "Product quality is poor, broke within 2 days of use"
8. "Neha, 35, Hyderabad, Rating:2"

1. Which city appears most?
2. What's the average rating?

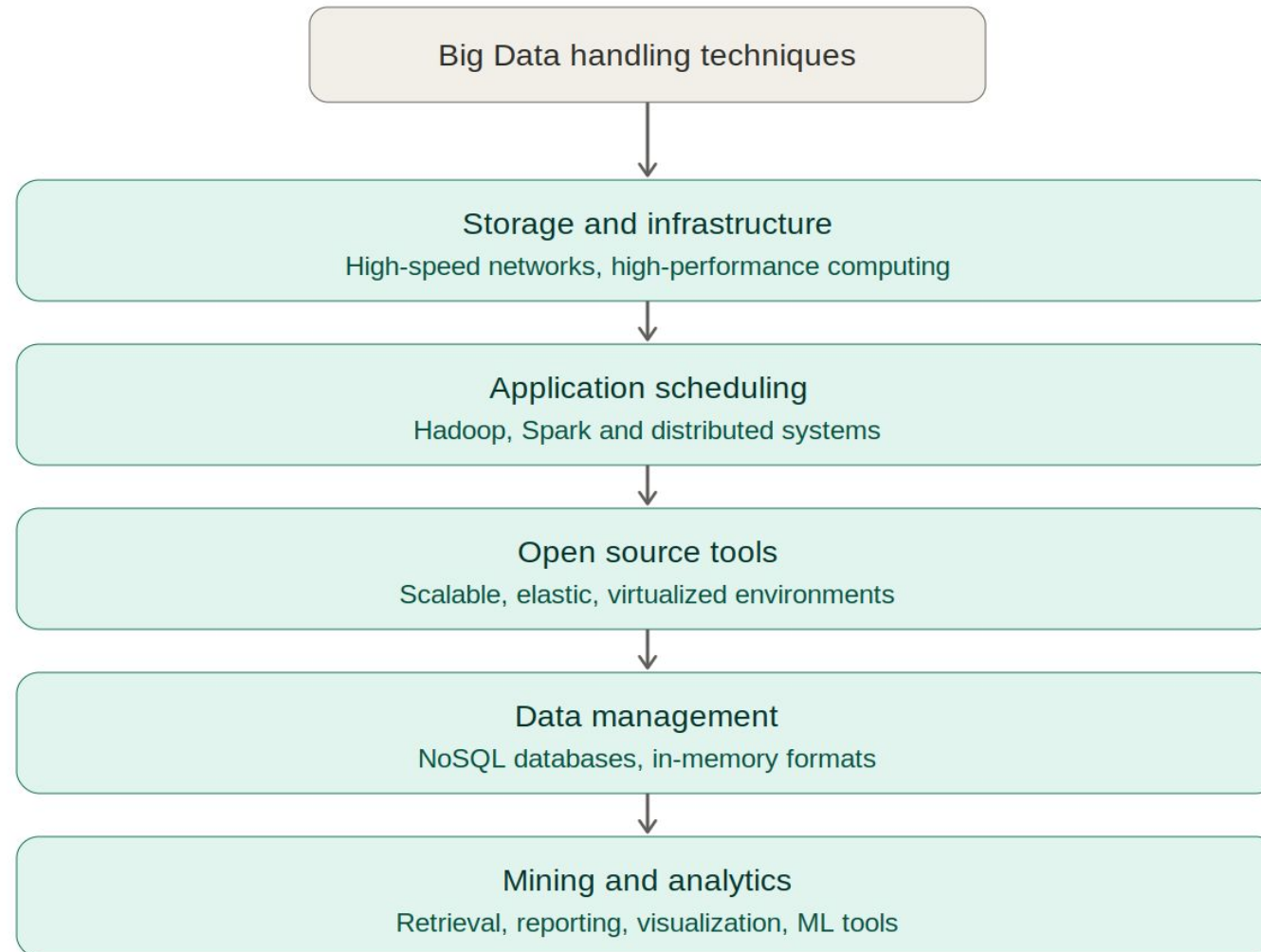
Big Data can be **classified on the basis of its characteristics** that are used for designing data architecture for processing and analytics. Table 1.1 gives various classification methods for data and Big Data.

Basis of Classification	Examples
Data sources (traditional)	Data storage such as records, RDBMs, distributed databases, row-oriented in-memory data tables, column-oriented in-memory data tables, data warehouse, server, machine-generated data, human-sourced data, Business Process (BP) data, Business Intelligence (BI) data
Data formats (traditional)	Structured and semi-structured
Big Data sources	Data storage, distributed file system, Operational Data Store (ODS), data marts, data warehouse, NoSQL database (MongoDB, Cassandra), sensors, audit trail of financial transactions, external data such as web, social media, weather data, health records
Big Data formats	Unstructured, semi-structured and multi-structured data

Data Stores structure	Web, enterprise or cloud servers, data warehouse, row-oriented data for OLTP, column-oriented for OLAP, records, graph database, hashed entries for key/value pairs
Processing data rates	Batch, near-real-time, real-time, streaming
Processing Big Data rates	High volume, velocity, variety and veracity, batch, near real-time and streaming data processing
Analysis types	Batch, scheduled, near real-time datasets analytics
Big Data processing methods	Batch processing (for example, using MapReduce, Hive or Pig), real-time processing (for example, using SparkStreaming, SparkSQL, Apache Drill)

Data analysis methods	Statistical analysis, predictive analysis, regression analysis, Mahout, machine learning algorithms, clustering algorithms, classifiers, text analysis, social network analysis, location-based analysis, diagnostic analysis, cognitive analysis
Data usages	Human, business process, knowledge discovery, enterprise applications, Data stores

Following are the techniques deployed for Big Data storage, applications, data management and mining and analytics:



Scalability & Parallel Processing

The Core Problem

Big Data means **huge datasets**: Example: terabytes to petabytes.

Processing this much data:

- Needs **serious computing power** (hundreds of computing nodes working together)
- Needs to happen **quickly**
- Needs to happen **cheaply**

Balancing all three (speed, scale, cost) is hard that's the challenge this chapter addresses.

Convergence of Data Environments and Analytics

Key idea: Big Data doesn't *replace* traditional data systems, it **works alongside them**.

- Traditional systems = RDBMS tables (regular databases) or data warehouses
- Big Data systems = need to grow in two directions to handle the load

What is Scalability? **Simple definition:** Scalability is a system's ability to **grow its capacity** (storage, processing, analytics power) to handle more work.

Think of it like this:

- If your workload increases, your system needs to be able to **grow with it**
- When the workload becomes too much for the current system to handle, you need to either:
 - **Scale up** (upgrade existing machines), or
 - **Scale out** (add more machines)

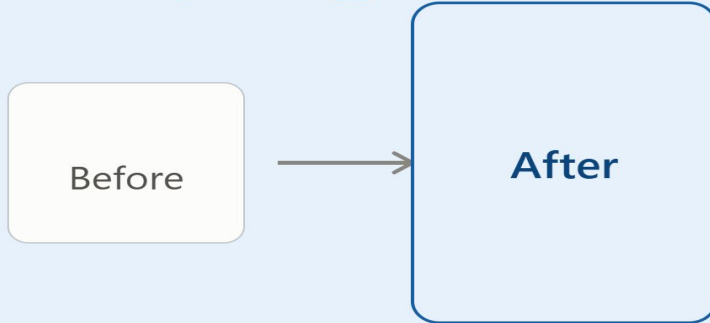
Analytics Scalability to Big Data

Refers to how well an analytics system, tool, or process can maintain (or improve) its performance, accuracy, and usability as the volume, velocity, and variety of data it handles grows, often into the "big data" range (terabytes to petabytes, high-speed streaming, diverse formats).

- 1. Vertical Scalability**
- 2. Horizontal Scalability**

Vertical scaling

Scale up: one bigger machine



More CPU, RAM, storage
on the same node



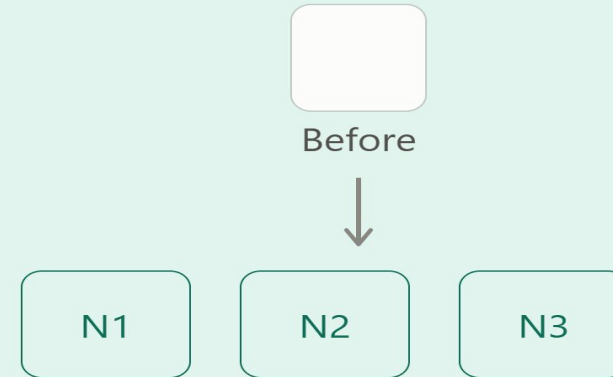
One super-efficient worker

doing the job of five

- + Simple to manage
- Hits a hardware ceiling

Horizontal scaling

Scale out: more machines



Parallel distributed nodes



Five separate workers

splitting the job at once

- + Scales almost limitlessly
- Needs node coordination (IPC)

- Easiest way to scale up and scale out execution of analytics software is to implement it on a bigger machine with more CPU's for greater Volume, Velocity and Variety and complexity of data.
- Alternative ways for scaling up and out processing of **analytics software** and **Big Data analytics** deploy the Massively Parallel Processing Platforms (**MPPs**), **cloud**, **grid**, **clusters**, and **distributed computing software**.

Computing methods for high availability and scalable computations and analysis

1. **MPPs**
2. **Cloud Computing**
3. **Cluster Computing**
4. **Grid Computing**
5. **Volunteer Computing**

1. Massively Parallel Processing Platforms

One worker is slow.
Many workers, at once,
are fast.

That's the entire idea behind Massively Parallel Processing (MPP) — split a huge task into small pieces, and let many processors solve them simultaneously instead of one processor solving them in a row.

THE PROBLEM

One computer isn't enough

Some jobs — like analyzing huge datasets — are too big or too complex for a single machine. It either takes forever, or the computer simply runs out of memory trying.

THE SOLUTION

Split it, then run it together

Break the big task into smaller pieces and hand them to many processors — or many computers — that all work at the same time. The total time drops dramatically.

That's the entire idea behind Massively Parallel Processing (MPP) — split a huge task into small pieces, and let many processors solve them simultaneously instead of one processor solving them in a row.

THE PROBLEM

One computer isn't enough

Some jobs — like analyzing huge datasets — are too big or too complex for a single machine. It either takes forever, or the computer simply runs out of memory trying.

THE SOLUTION

Split it, then run it together

Break the big task into smaller pieces and hand them to many processors — or many computers — that all work at the same time. The total time drops dramatically.

Watch the difference

Same job — 8 tasks to complete. One track does them one-by-one. The other splits them across 8 workers.

Three levels of "splitting the work"

1. **Same CPU, different threads** – like one worker using both hands to do two small jobs at once.
2. **Same computer, different CPUs** – like having multiple workers in the same office.
3. **Different computers entirely** – like having workers in different offices, all connected, working on the same overall project.

Example for MPP: Distributed Computing Model

A **distributed computing model** is when you spread big data processing across many connected computers (nodes) instead of relying on one machine and those computers work together as if they were one system.

Bunch of computers could be in a **cloud**, a **grid**, or a **cluster** all linked together by fast networks. Together they store and process large datasets that no single machine could handle alone.



CLOUD

Rented, far apart

Computing power rented over the internet from a provider's data centers — you don't know or care exactly where the machines sit.

Location → scattered, provider-owned

Coupling → loose, on-demand

Example → AWS, Azure, GCP



GRID

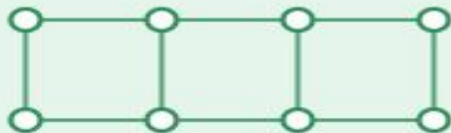
Pooled, across organizations

Independent computers — often owned by different institutions — voluntarily pool spare resources over a network to tackle one shared problem.

Location → multiple sites/organizations

Coupling → loosely coordinated

Example → research/scientific grids



CLUSTER

Owned, same room

A tightly packed group of machines in one location, wired together over a fast local network, acting as a single system.

Location → one site, same racks

Coupling → tight, high-speed link

Example → Hadoop/Spark cluster

Table 1.2 Distributed computing paradigms

Distributed computing on multiple processing nodes/clusters	Big Data > 10 M	Large datasets below 10 M	Small to medium datasets up to 1 M
Distributed computing	Yes	Yes	No
Parallel computing	Yes	Yes	No
Scalable computing	Yes	Yes	No
Shared nothing (No in-between data sharing and inter-processor communication)	Yes	Limited sharing	No
Shared in-between between the distributed nodes/clusters	No	Limited sharing	Yes

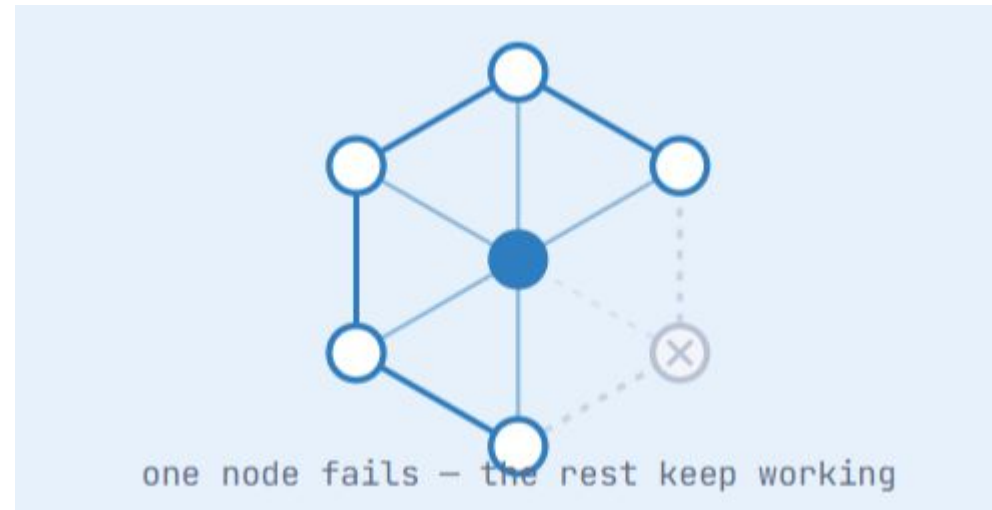
Table 1.2 gives the requirements of processing and analyzing big, large and small to medium datasets on distributed computing nodes. Big Data processing uses a parallel, scalable and no-sharing program model, such as MapReduce, for computations on it.

Shared nothing : each node has its own memory and disk, and doesn't constantly talk to other nodes or share data mid-task. Every node works independently on its own slice of the problem.

2. Cloud Computing

"Cloud computing is a type of Internet-based computing that provides shared processing resources and data to the computers and other devices on demand."

Why cloud is good for big data?



Five defining features

What actually makes something "cloud computing" rather than just a rented server.

01



On-demand service

Spin up resources instantly, whenever you need them.

02



Resource pooling

Many users share the same underlying hardware pool.

03



Scalability

Grow or shrink capacity smoothly as demand changes.

04



Accountability

Usage is tracked and measured, so it's billed fairly.

05



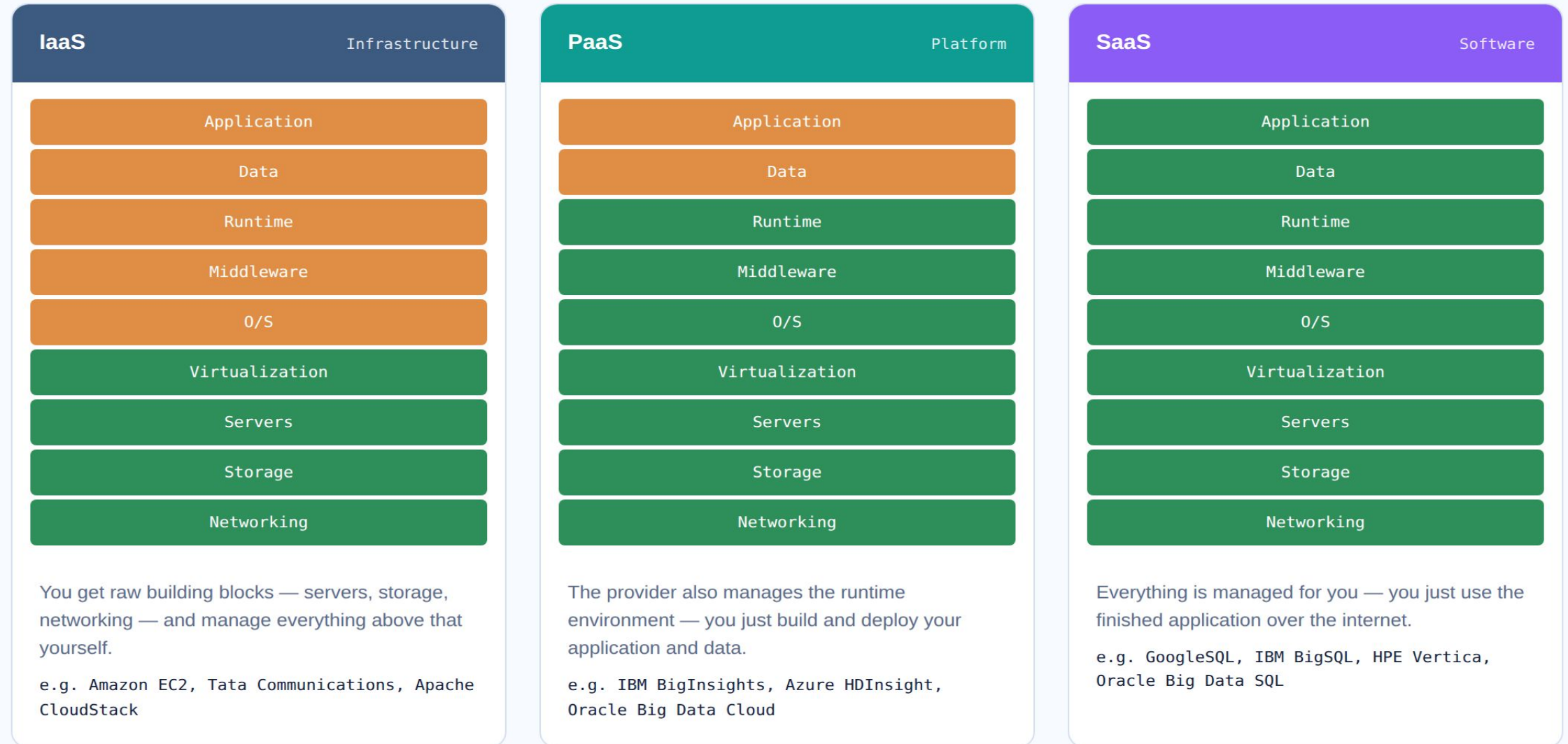
Broad network access

Reachable from anywhere, on any device, anytime.

The three service types

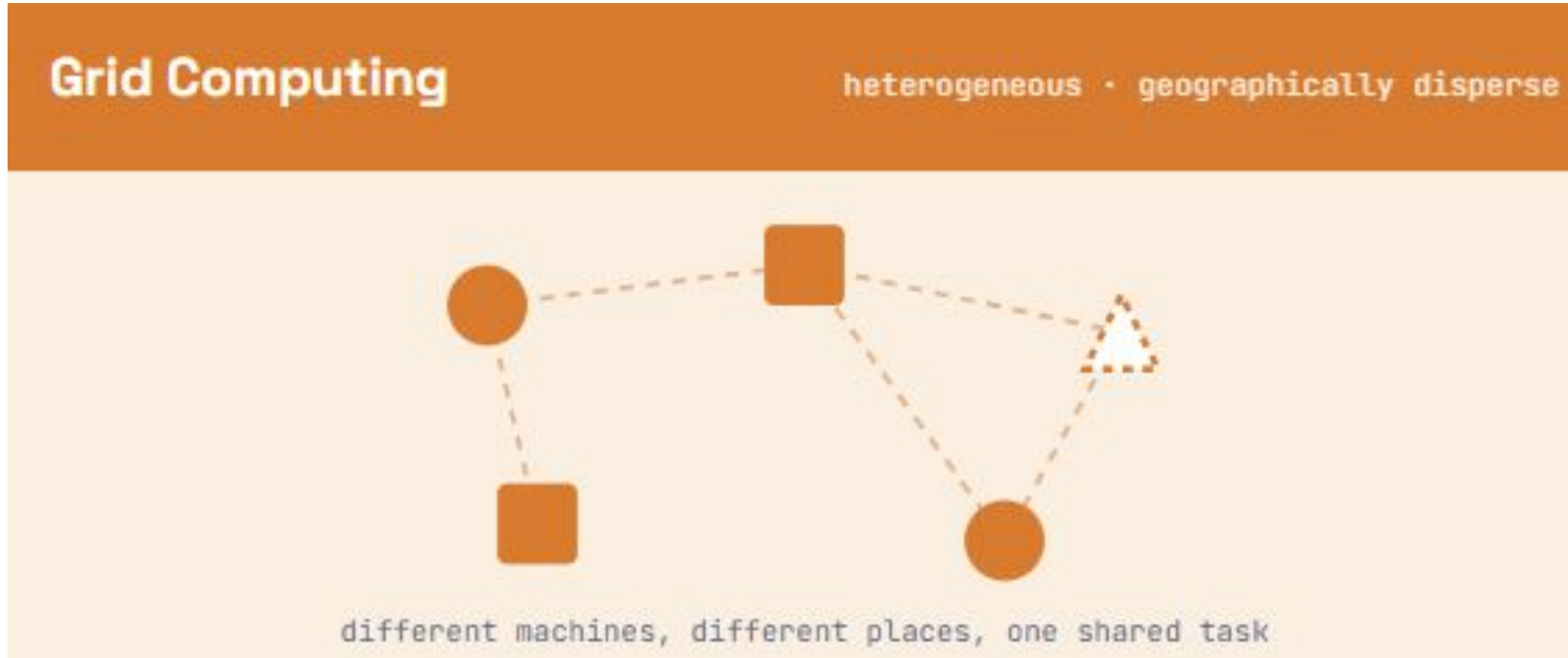
The difference between IaaS, PaaS, and SaaS is simply: **how much the provider manages for you.**

■ you manage ■ provider manages



3. Grid Computing

Grid Computing refers to distributed computing, in which a group of computers from several locations are connected with each other to achieve a common task.



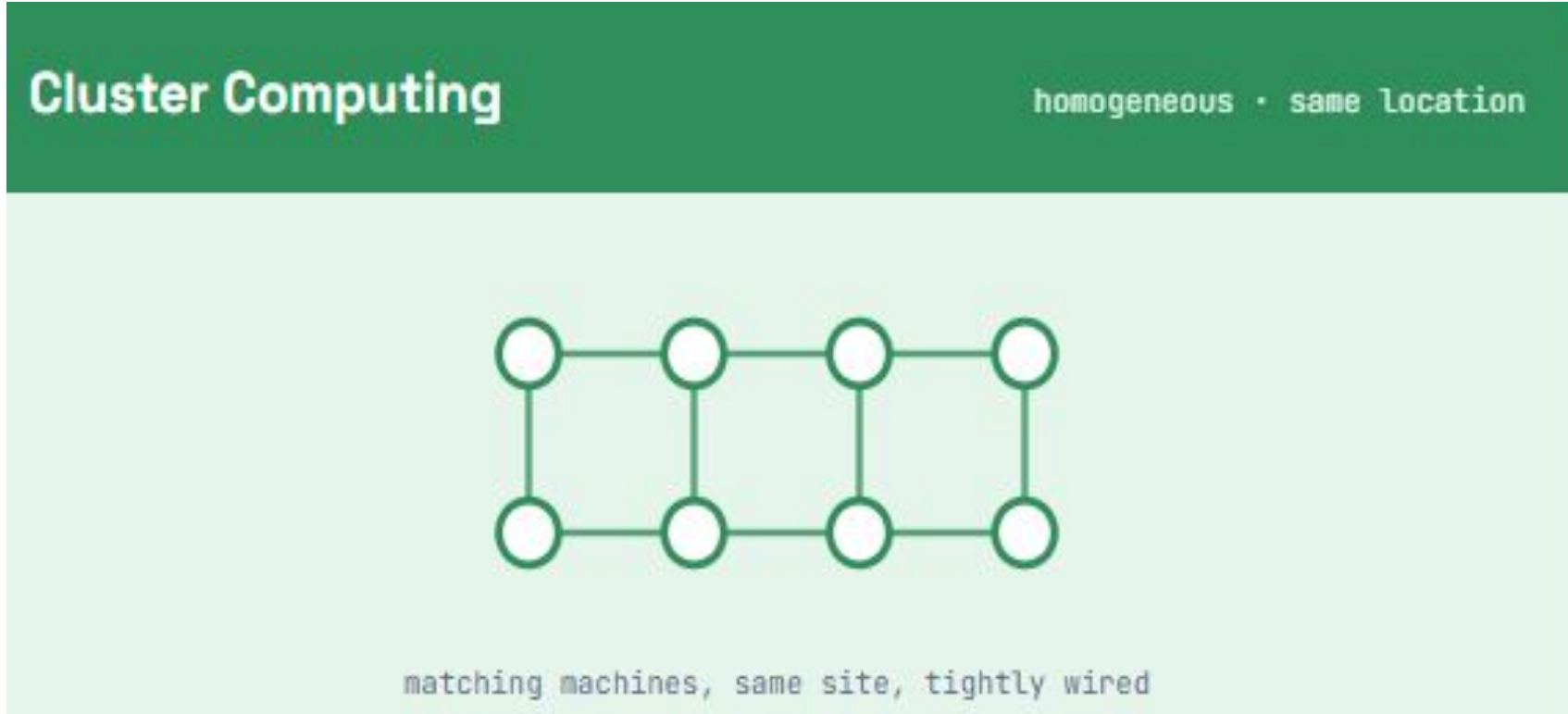
The machines don't have to match — grids are **heterogeneous**.

BEST SUITED FOR

- Large-scale, data-intensive storage (not lots of tiny files)
- Computationally intensive work (the "computational grid")
- Flexible, coordinated resource sharing across users

Drawback: the grid is a single point of failure if a participating node underperforms or drops out, the whole task can suffer.

4. Cluster Computing: A group of similar computers connected by a network, working together on the same task from one location. Clusters are homogeneous and tightly coupled.



Clusters are used mainly for **load balancing**. They shift processes between nodes to keep an even load on the group of connected computers. Hadoop architecture uses the similar methods



Figure 3: Load Balancing

Table 1.3 Grid computing and related paradigms

Distributed computing	Cluster computing	Grid computing
• Loosely coupled • Heterogeneous • Single administration	• Tightly coupled • Homogeneous • Cooperative working	• Large scale • Cross organizational • Geographical distribution • Distributed management

5. Volunteer computing is a form of distributed computing where ordinary people (or organizations) donate their personal computers' spare processing power to help run large-scale projects usually scientific research.

How it works?

- A project (often run by a university or research institution) needs massive computing power to solve a big problem
- Instead of buying expensive supercomputers, they ask the public to **volunteer** their computers' idle time
- Volunteers install software that runs small chunks of the project's computation in the background while they're not otherwise using their CPU
- Results get sent back to the project, which stitches everything together

Data Architecture Design

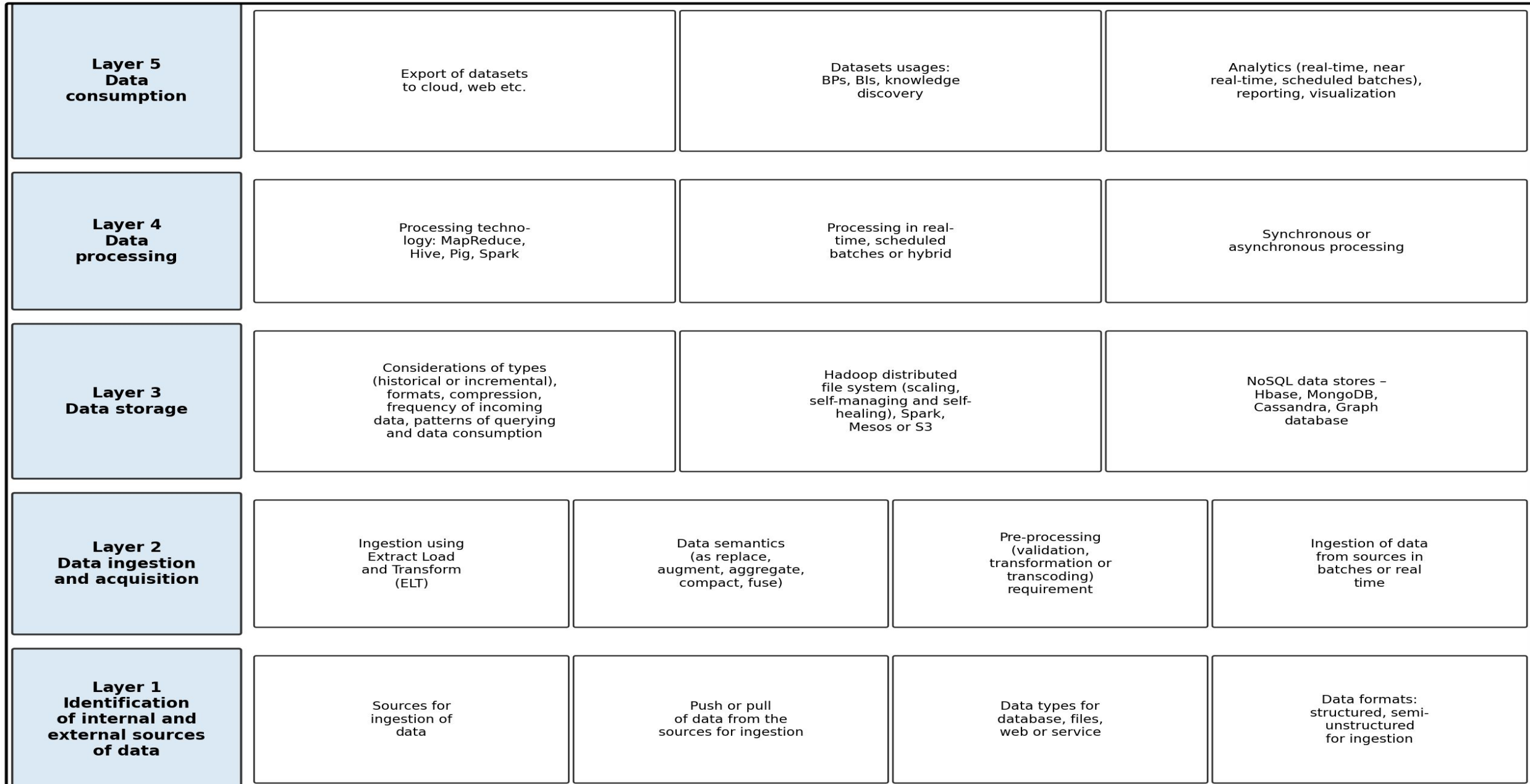
Big Data Architecture is basically the **blueprint** for how a company handles huge amounts of data where it comes from, how it's collected, where it's stored, how it's processed, and how people finally use it.

Architecture logically defines how Big Data solution will work, the core components (hardware, database, software, storage) used, flow of information, security and more."

Data analytics need the number of **sequential steps**.

Big Data architecture design task simplifies when using the **logical layers approach**. (Figure 1.2) shows the logical layers and the functions which are considered in Big Data architecture.

Figure 1.2 Design of logical layers in a data processing architecture, and functions in the layers



Managing Data for Analysis

- Data managing means **enabling, controlling, protecting, delivering** and enhancing the value of data and information asset.
- Reports, analysis and visualizations need well-defined data. Data management also enables data usage in applications.
- The process for managing needs to be well defined for fulfilling requirements of the applications.

Data Management Functions

14 core functions grouped by purpose

Protect & Secure

1 Data assets creation, maintenance & protection

5 Data security, access control & privacy

✓ Govern & Quality

2 Data governance: availability, usability, integrity, security

6 Managing data quality

Structure & Organize

3 Data architecture, modelling & analysis

4 Database maintenance & administration (RDBMS, NoSQL)

8 Managing documents, records & contents

9 Reference & master data creation, control

Integrate & Move

7 Data collection using ETL process

10 Data & application integration

11 Integrated data mgmt, fast access, automation

Analyze & Leverage

12 Data warehouse management

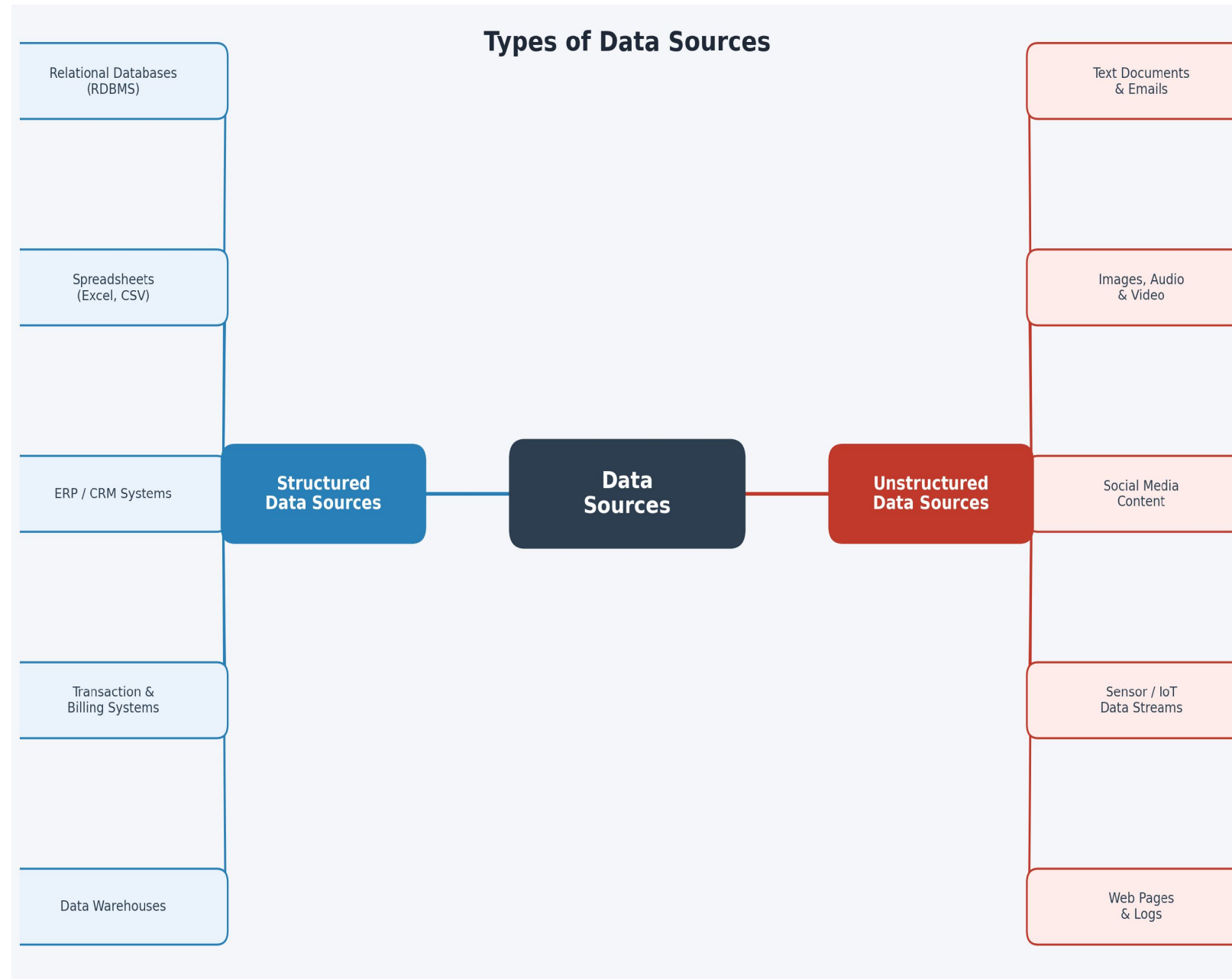
13 Maintenance of business intelligence

14 Data mining & analytics algorithms

Data Sources, Quality, Pre-Processing & Storing

Data Sources

- Applications, programs and tools use data. Sources can be *external*, such as sensors, trackers, web logs, computer systems logs and feeds. Sources can be machines, which source data from data-creating programs.
- Data sources can be structured, semi-structured, multi-structured or unstructured.
- Data sources can be social media (L1 in Figure 1.2).
- A source can be *internal*. Sources can be data repositories, such as database, relational database, flat file, spreadsheet, mail server, web server, directory services, even text or files such as comma-separated values (CSV) files.
- Source may be a data store for applications (L4 in Figure 1.2).



1. Structured Data Sources

Structured data sources are data that follows a fixed, organized format like rows and columns in a table (think Excel or a database). This section explains *where* this kind of data comes from and *how systems identify it*.

◆ Where structured data lives?

It can come from:

- **A file** (like a CSV or Excel file)
- **A database** (like SQL Server, MySQL, Oracle, IBM DB2)
- **Streaming data** (data flowing in continuously, e.g., live transactions)

It might sit on the **same computer** running the program, or on a **separate networked computer/server**.

Examples given: SQL Server, MySQL, Microsoft Access, Oracle DBMS, IBM DB2, Informix, Amazon SimpleDB, or even just a folder of files on a server.

❖ Every data source needs a name

Just like you'd label a folder on your desktop, every data source needs a **clear, meaningful name** so systems/processes can find and use it correctly.

Example: If you're storing student grades, instead of naming it "data1," you'd name it something meaningful like:

StudentName_Data_Grades

❖ The "Data Dictionary" — a master directory

- A **data dictionary** is like a **phone book for data sources**. It: Keeps a list of all data sources ("lookup tables")
- Is stored in **one central place** (not scattered everywhere)
- Makes it easy to **find sources** and **manage changes** without hunting around
- **Example name:** **UniversityStudents_DataPlusGrades**
- The server that manages this master directory is sometimes called a **NameNode**.

Microsoft applications consider two types of sources for processing:
(i) machine sources and (ii) file sources.

Oracle applications consider two types of data sources:
(i) database, which identifies the database information that the software needs to connect to database, and (ii) logic-machine, which identifies the machine which runs batches of applications and master business functions.

2. Unstructured Data Sources

Unstructured data is data that **doesn't fit neatly into rows and columns** like a database table. It's messy, free-form, and travels across networks fast, so systems need to process it quickly ("high velocity").

- **Where it comes from?:** Distributed file systems (data spread across many servers, not one central database)
- **Common file types:** `.txt` (plain text), `.csv` (comma-separated files)
- **Key-value pairs:** Data stored like a dictionary — e.g., `"name": "John"` (similar to hash tables)

Has some internal structure, but no formal model:

- Emails *do* have parts (subject, sender, body) but there's no strict database-style schema
- Facebook posts, Tweets have some structure (timestamp, author, text) but aren't relational tables

Data Sources: Sensors, Signals & GPS

Physical devices that sense the real world and generate data automatically

Types of Sources

- Sensors & sensor networks
- Signals from machines, devices, controllers
- Edge nodes (e.g. smart thermostat)
- GPS systems
- M2M – Machine-to-Machine communication

What Sensors Measure

- Temperature, pressure, humidity
- Light intensity
- Traffic / proximity of objects
- Acceleration & location
- Orientation & magnetic intensity
- ✓ **Key role in automotive industry (self-driving cars, safety systems)**

RFID

- Radio Frequency Identification
- Tracks parcels, goods & deliveries
- Common in supply chain management
- e.g. tracking a package from warehouse to delivery



Smart Sensors (IoT)

- Embedded in processors with machine-learning built in
- Wireless communication capability
- Devices can "think" a little & talk to each other
- Power IoT applications: smart homes, wearables, industrial monitoring

Data Quality

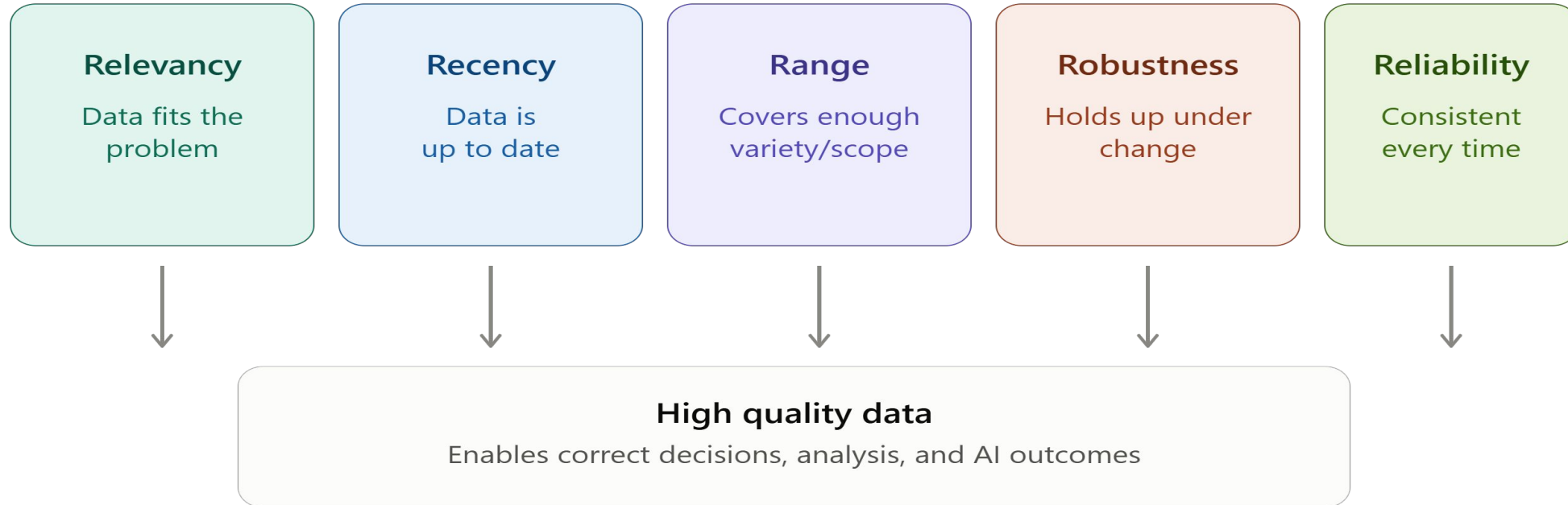
Data quality is what when it represents the real-world construct to which references are taken. High quality data, which enables all the required operations, analysis, decisions, planning and knowledge discovery correctly

Does your data actually reflect the real thing it's supposed to represent?

- For example, if you're recording someone's age, height, or sales numbers*
- Good quality data means those numbers are correct, accurate, and truly match reality.*

The 5 R's of Data Quality (especially important for AI)

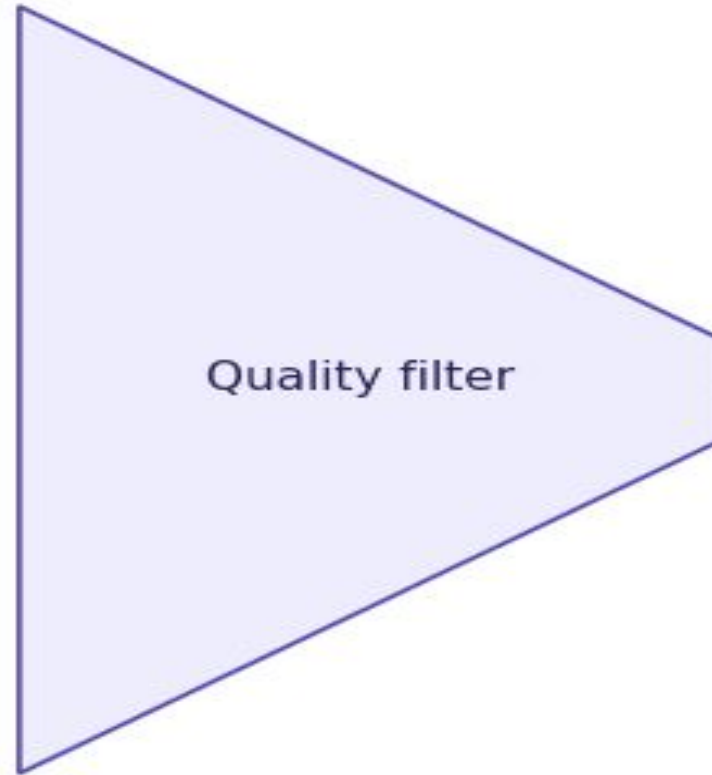
The five R's of high quality data



Huge raw volume



High quality data



Clean dataset

Quality of Data: Why is it hard to define?

- *There's no single fixed rule that says "this data is good" or "this data is bad."*
- *Instead, people set certain conditions or benchmarks (either numeric values or descriptive qualities), and based on whether the data meets those conditions, we say the data quality is **high** or **low**.*

Data Integrity

Refers to the maintenance of consistency and accuracy in data over its usable life. Data should be incorruptible

Example: *The data you stored is the same data you get back.* If a professor enters a student's grade as 85, that 85 should still say 85 **next month, next year, no matter how many times the software touches it.** If any process a bug, a bad update, a crash mid-write silently changes 85 to 58, integrity is broken.

Best example: A hospital patient's blood type is recorded as "O+". If a database sync error flips it to "O-", the record is now corrupted and this isn't a small error, it's dangerous. Data integrity means that never happens; the system guarantees the value stays exactly as entered until someone deliberately and correctly changes it.

Data Noise, Outliers, Missing & Duplicate Values

These are the four common ways real-world data gets "dirty" meaning it stops accurately representing reality.

Problem	Simple meaning	Best example
Noise	Small, random errors mixed into otherwise correct data — like static on a radio	A thermometer reads 36.9°C, 37.1°C, 36.8°C for the same steady body temperature — tiny random fluctuations from sensor imprecision, not real changes
Outlier	A value way outside the normal/expected range	A student's exam score is entered as 950 out of 100 — clearly a typo (missing a decimal, e.g. 95.0), way outside the valid range
Missing Value	Data that should be there but isn't	A survey form has an "age" field left completely blank because the respondent skipped it
Duplicate Value	The same record appears more than once	The same customer "John Smith, john@mail.com " is entered twice in a CRM because he filled the signup form twice

Student	Grade
Asha	85 (was 82.6)
Rahul	950
Meera	(empty)
Vikram	78
Vikram (dup)	78

○ Noise — tiny random fluctuation

○ Outlier — impossible value

○ Missing — no value entered

○ Duplicate — same record twice

Example 1.8

Data Pre-processing

- Data pre-processing is an important step at the ingestion layer (Figure 1.2). For example, *consider grade point example in previous data . The outlier needs to be removed.*
- Pre-processing is a must before data mining and analytics. Pre-processing is also a must before running a Machine Learning (ML) algorithm. Analytics needs **prior screening** of data quality also. Data when being exported to a cloud service or data store needs pre-processing.

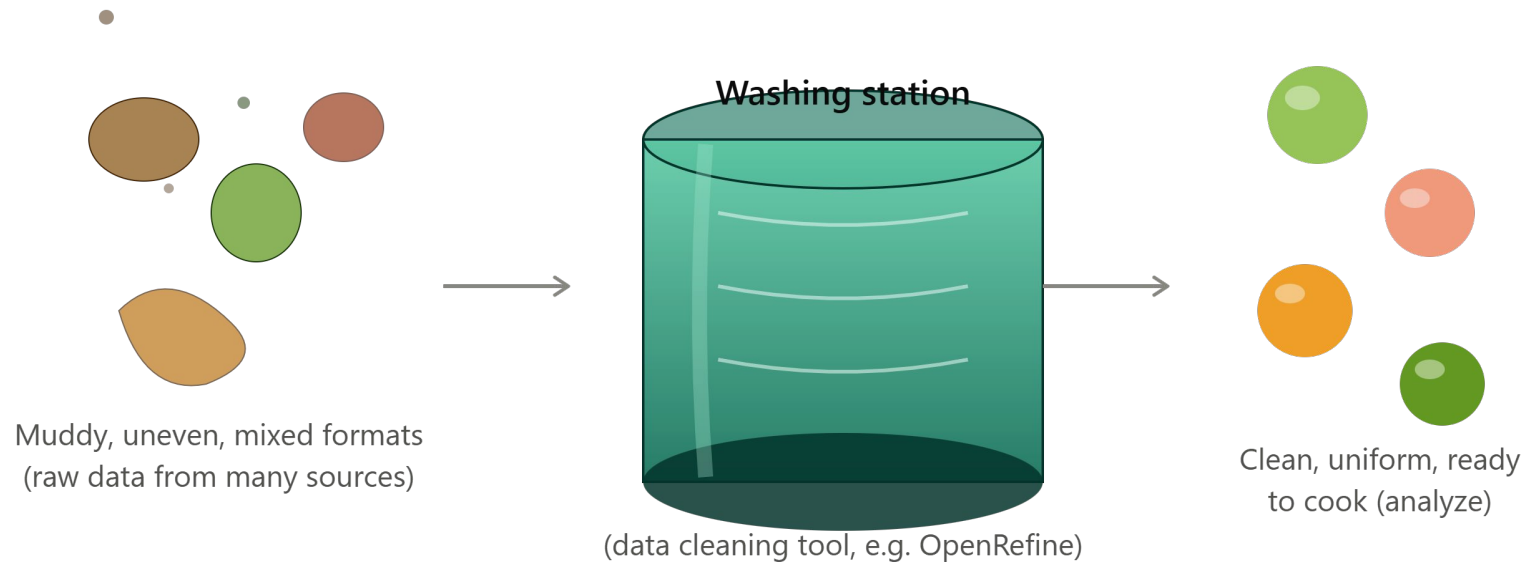
***Data pre-processing** is the step in the data lifecycle (at the ingestion layer) where raw data is cleaned up and prepared before it's used for data mining, analytics, or machine learning.*

Pre-processing needs are:

- (i) Dropping out of range, inconsistent and outlier values**
- (ii) Filtering unreliable, irrelevant and redundant information**
- (iii) Data cleaning, editing, reduction and/or wrangling**
- (iv) Data validation, transformation or transcoding**
- (v) ELT processing**

Data Cleaning

Data cleaning refers to the process of removing or correcting incomplete, incorrect, inaccurate or irrelevant parts of the data after detecting them, in Example 1.8 correcting the grade outliers or mistakenly entered values means cleaning and correcting the data.

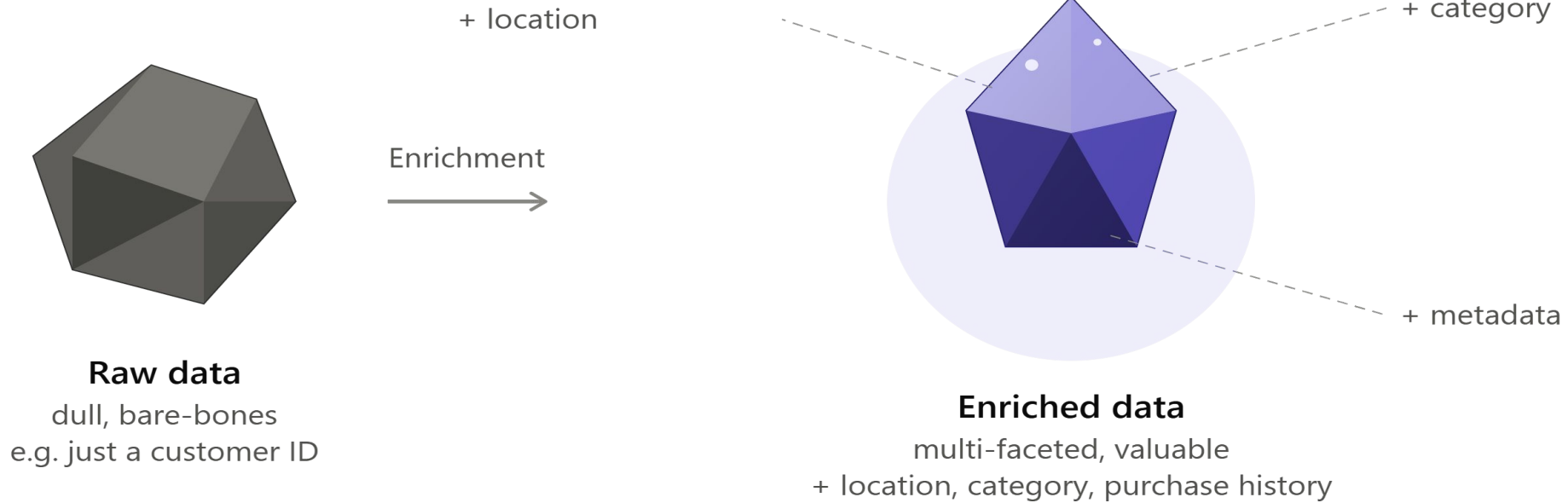


Vegetable cleaning process from different sources

Data Cleaning Tools Data cleaning is done before mining of data. Incomplete or irrelevant data may result into misleading decisions. It is not always possible to create well-structured data. Data can generate in a system in many formats when it is obtained from the web. Data cleaning tools help in refining and structuring data into usable data. Examples of such tools are **OpenRefine** and **DataCleaner**.

Data Enrichment

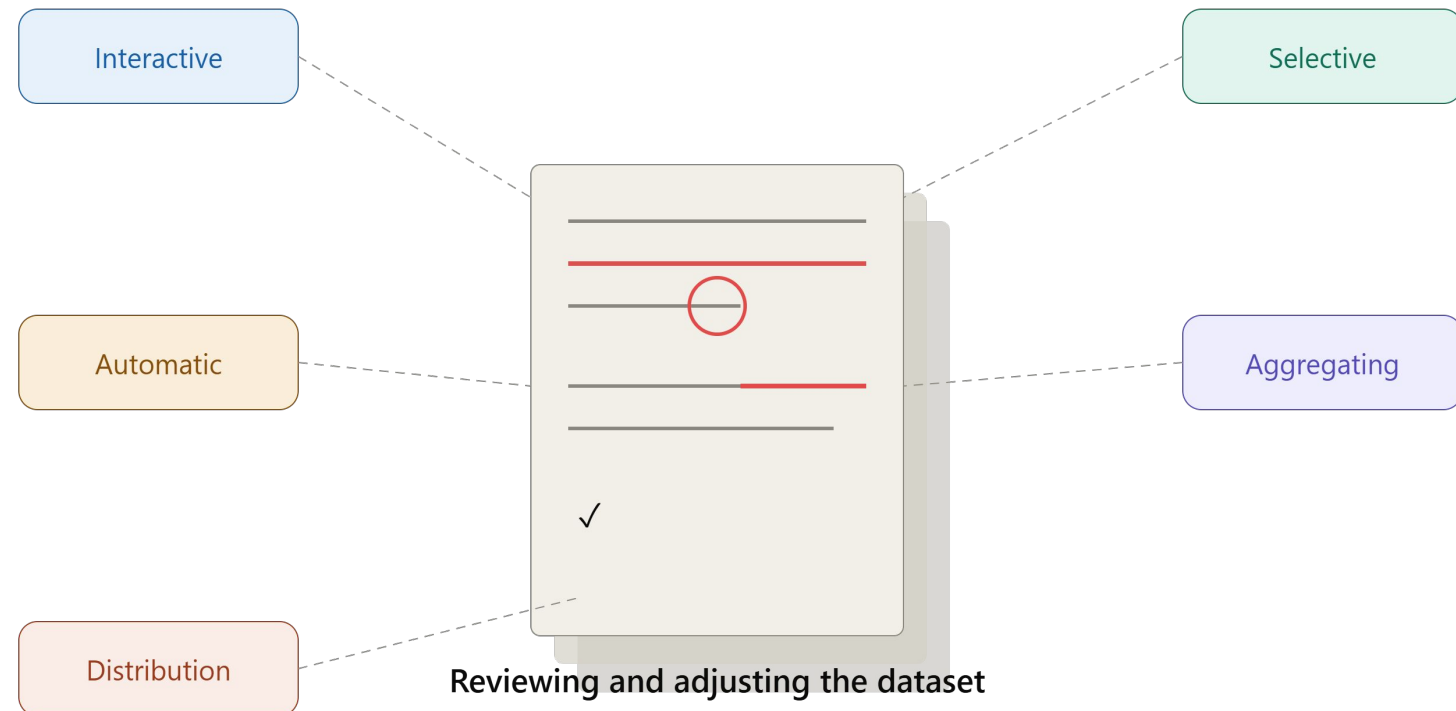
"Data enrichment refers to operations or processes which refine, enhance or improve the raw data."



Data Editing - "The Proofreading Pass"

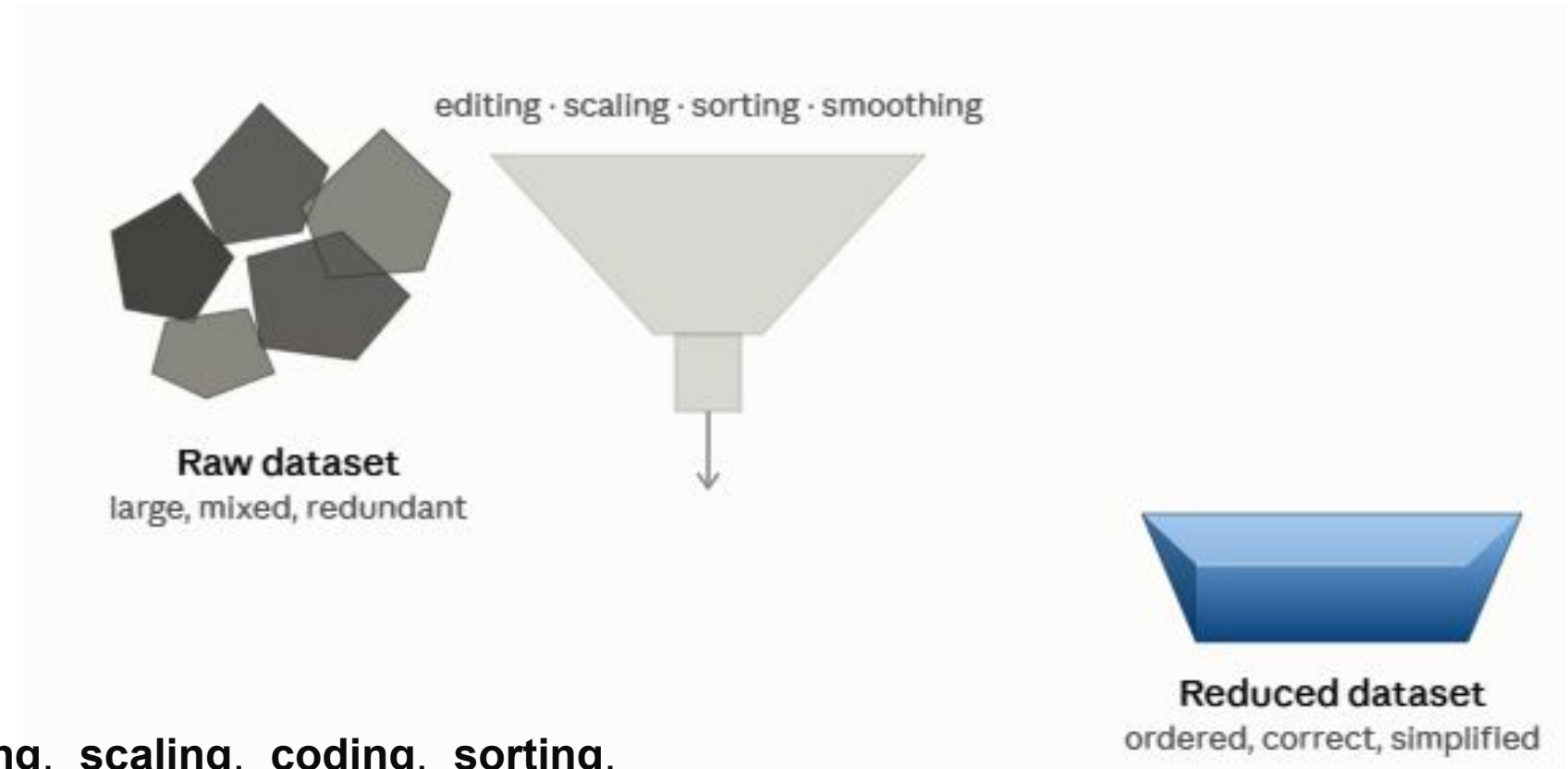
Editing methods are:

- (i) Interactive
- (ii) Selective
- (iii) Automatic
- (iv) Aggregating
- (v) Distribution.



Data Reduction

Data reduction enables the transformation of acquired information into an ordered, correct and simplified form. The reductions enable ingestion of meaningful data in the datasets. The basic concept is the reduction of multitudinous amount of data, and use of the meaningful parts.

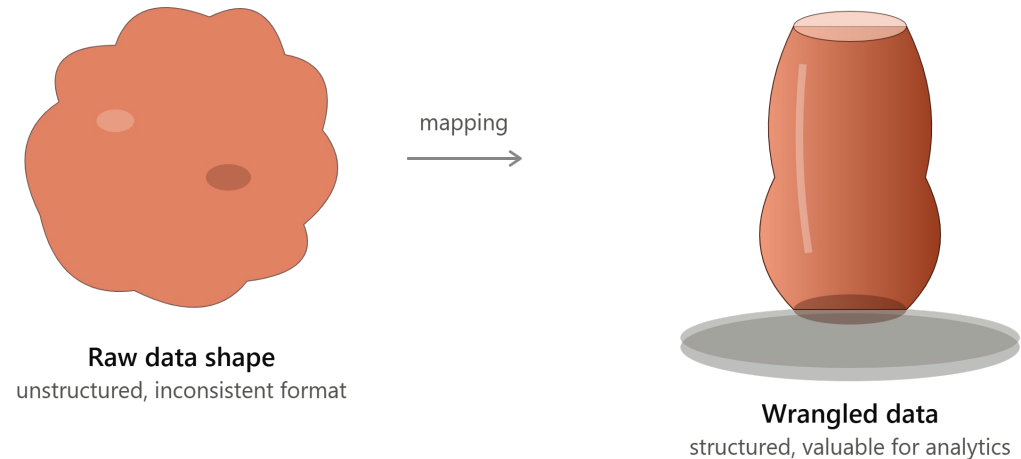


The reduction uses **editing, scaling, coding, sorting, collating, smoothening, interpolating** and **preparing tabular summaries**.

Data Wrangling

Like "molding clay into a new shape"

Data wrangling refers to the **process of transforming and mapping the data**. Results from analytics are then appropriate and valuable. For example, mapping enables data into another format, which makes it valuable for analytics and data visualizations.



Data Format used during Pre-Processing

Examples of formats for data transfer from (a) data storage, (b) analytics application, (b) service or (d) cloud can be:

for example, in the form of tables

- (i) Comma-separated values CSV (example 1.9)
- (ii) Java Script Object Notation (JSON) as batches of object arrays or resource arrays
- (iii) Tag Length Value (TLV)
- (iv) Key-value pairs
- (v) Hash-key-value pairs

CSV Format

An example is a data record. Each record consists of one or more fields, separated from each other by commas. RFC 4180 standard specifies the various specifications. A CSV file may also use space, tab or delimiter tab-separated formats for the values in the fields. This is a loose terminology. The following example explains the conversion process.

EXAMPLE 1.9

Consider the example of a table in a grade sheet. A CSV file is easily understandable when the table's first row specifies the column heads. Three columns of the first row are Subject Code, Subject Name and Grade and three columns of the second row are CS101, "Theory of Computations" and 7.8, as shown below:

Subject Code	Subject Name	Grade
CS101	"Theory of Computations"	7.8
CS102	"Computer Architecture"	7.2
-	-	-

SOLUTION

The first and second lines in the CSV file are:

Subject Code, Subject Name, Grade

CS101, ""Theory of Computations"", 7.8.

CS102, ""Computer Architecture"", 7.8.

The two consecutive double-quotes mean that one of the double quotes is retained in the text "Theory of Computations". That one specifies that characters are inside the double quotes and represent a string.

Data Format Conversions

Transferring the data may need pre-processing for data-format conversions. Data sources store need portability and usability. A number of different applications, services and tools need a specific format of data only. Pre-processing before their usages or storage on cloud services is a must.

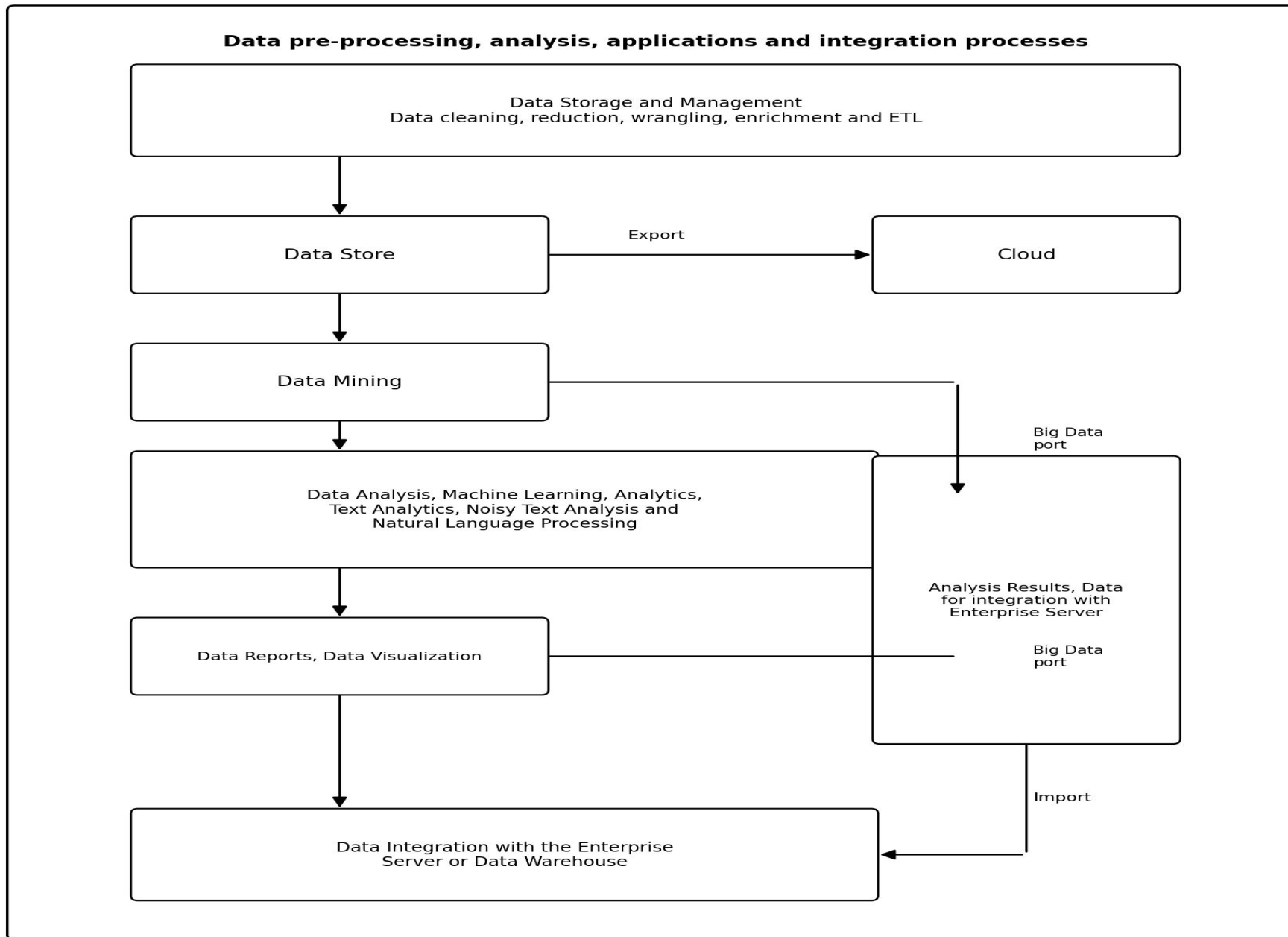


Figure 1.3: Data pre-processing, analysis, visualization, data store export

Cloud Services

- Cloud offers various services. (Section 1.3.3)
- These services can be accessed through a cloud client (client application), such as a web browser, SQL or other client.
- Figure 1.4 shows data-store export from machines, files, computers, web servers and web services.
- **The data exports to clouds**, such as IBM, Microsoft, Oracle, Amazon, Rackspace, TCS, Tata Communications or Hadoop cloud services.

Refer Example 1.10

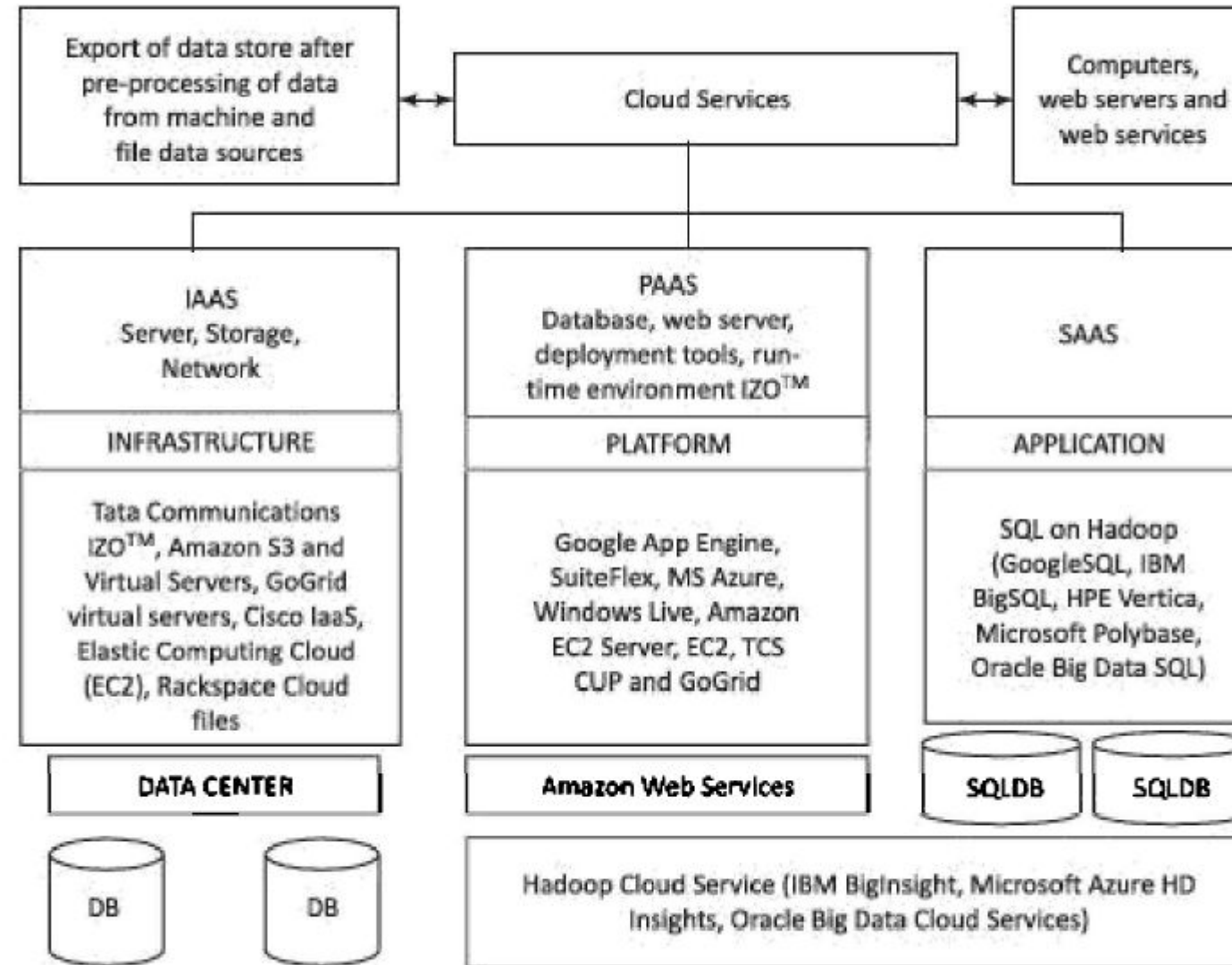


Figure 1.4 Data store export from machines, files, computers, web servers and

Google Cloud Platform

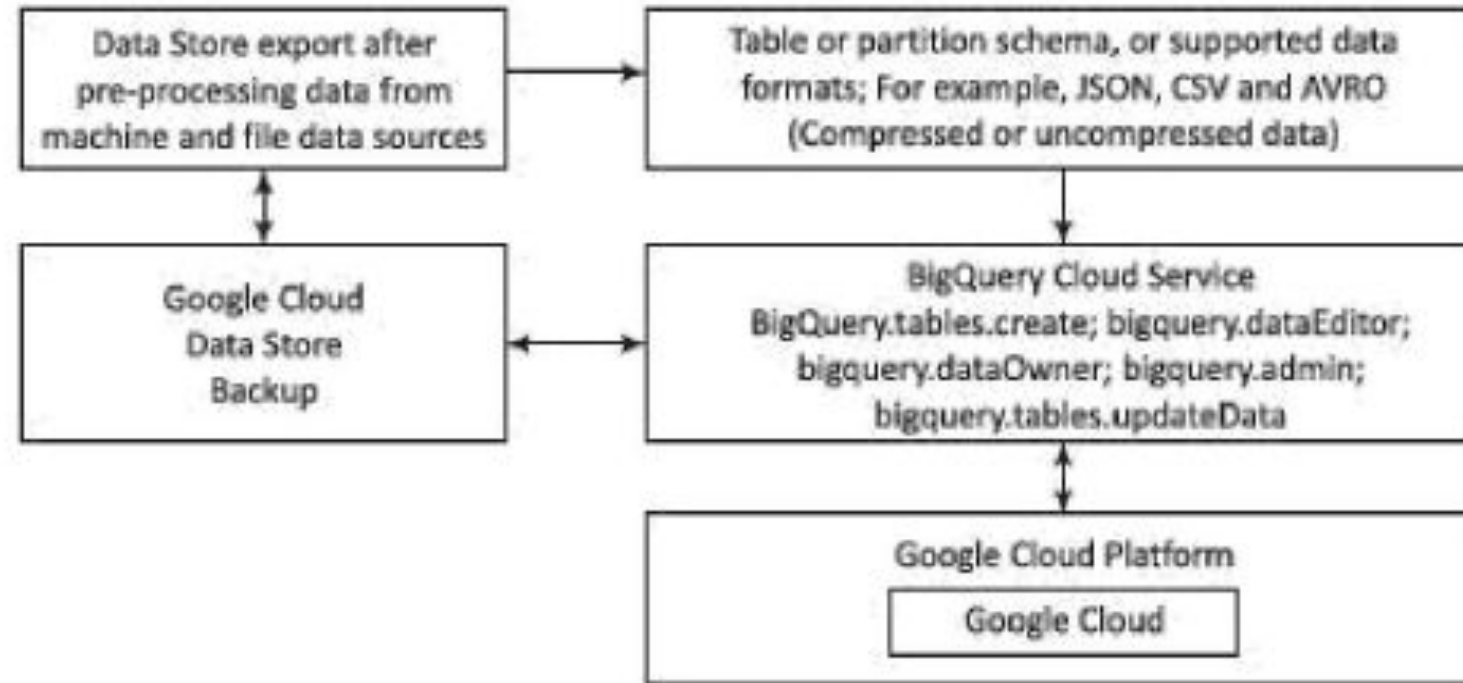


Figure 1.5 BigQuery cloud service at Google cloud platform

Data Storage & Analysis- Data Storage & Management Traditional Systems

Data Store with Structured or Semi-Structured Data

1. SQL Structured Query Language (SQL) is a domain-specific, standardized programming language used to interact with relational database management systems (RDBMS) such as MySQL, SQL Server, IBM Db2, PostgreSQL and Oracle Database.

An RDBMS uses SQL (Structured Query Language). SQL is a language for viewing or changing (update, insert or append or delete) databases. It is a language for data access control, schema creation and data modifications.

SQL was originally based on the tuple relational calculus and relational algebra. SQL can embed within other languages using SQL modules, libraries and pre-compilers. SQL does the following:

- **Create schema**, which is a structure which contains description of objects (base tables, views, constraints) created by a user. The user can describe the data and define the data in the database.
- **Create Catalog**, which consists of a set of schemas which describe the database.
- **Data Definition Language (DDL)** for the commands which depicts a database, that include creating, altering and dropping of tables and establishing the constraints. A user can create and drop databases and tables, establish foreign keys, create view, stored procedure, functions in the database etc.
- **Data Manipulation Language (DML)** for commands that maintain and query the database. A user can manipulate (INSERT/UPDATE) and access (SELECT) the data.
- **Data Control Language (DCL)** for commands that control a database, and include administering of privileges and committing. A user can set (grant, add or revoke) permissions on tables, procedures and views.

Large Data Storage using RDBMS

The Basic Idea

RDBMS (Relational Database Management System) stores large amounts of data in a **structured, table-like format** rows and columns, just like a spreadsheet. This structure makes it easy to organize, search, and manage huge volumes of data.

Why "Large" Data Needs Special Handling?

When data grows very large, it's not just about storing it, it's about doing it **safely, efficiently, and from many different sources**. So RDBMS systems handle several extra jobs:

1. Privacy & Security
2. Data Integration
3. Compaction
4. Fusion

Where the Data Comes From?

Large-scale data storage systems pull data from three main sources:

1. Machine-generated data
2. Human-sourced data
3. Business Process (BP) & Business Intelligence (BI) data

Distributed Database Management System (DDBMS):

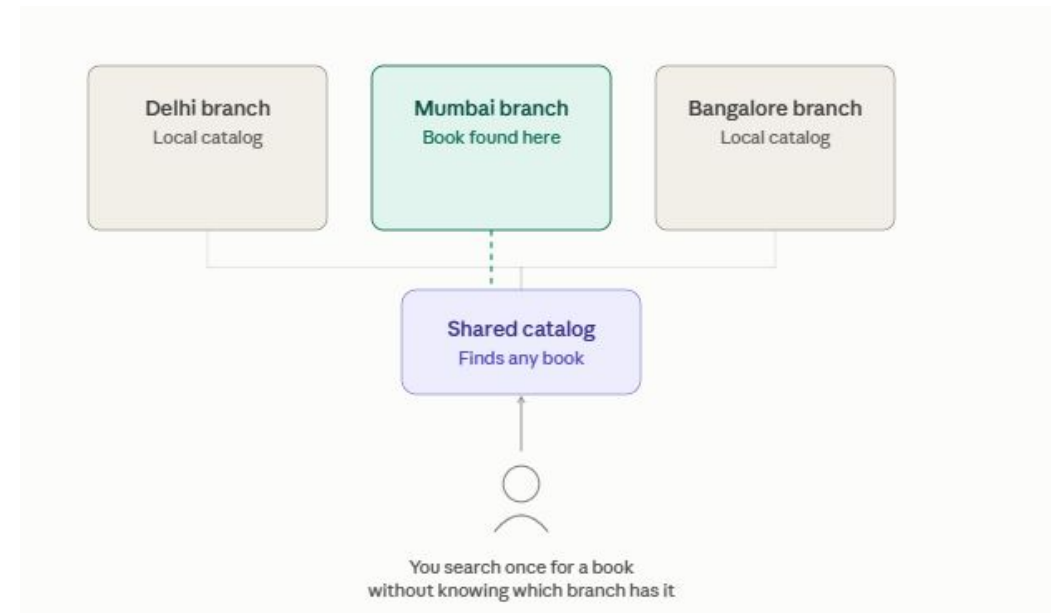
The Basic Idea

Instead of storing all your data in **one single place**, a Distributed Database spreads the data across **multiple computers/systems**, which are all connected through a network but to the user, it still *feels* like one single database.

A distributed DBMS (DDBMS) is a collection of logically interrelated databases at multiple system over a computer network. The features of a distributed database system are:

1. A collection of logically related databases.
2. Cooperation between databases in a transparent manner.
3. Should be 'location independent' which means the user is unaware of where the data is located, and it is possible to move the data from one physical location to another without affecting the user.

Example: Think of it like a chain of libraries in different cities, all sharing one Catalog system. You can search for a book from any branch, and the system finds it for you you don't need to know which branch actually has it.



In-Memory Column Formats

The Basic Idea

Normally, data in a table is stored **row by row** (like reading a spreadsheet left to right, one row at a time). But **columnar format** stores data **column by column** instead all values of one column are kept together in memory.

Why This Matters?

When you're doing analysis, you often only need **a few columns**, not the whole row.

Example: Imagine a big table of sales data with columns: **Name, Age, City, Product, Price, Date, Quantity... (20 columns)**

If you just want to know "**What's the average price?**", you only need the Price column not all 20 columns

- **Row format:** To read all prices, the computer has to jump through every row, skipping past Name, Age, City, etc. each time — **slow and wasteful**
- **Column format:** All the `Price` values are already sitting together in memory. The computer grabs them in one smooth sweep — **fast**

The Memory Access Trick

- In columnar format, values in the same column are stored at **consecutive (back-to-back) memory addresses**.
- So once the computer accesses the first value, getting the *next* value is just "move to the next address" very fast.
- In row format, the next value of the same column is **far away** in memory (after all the other columns of that row), so the computer has to calculate a bigger jump slower.
- Online Analytical Processing (OLAP) in real-time transaction processing is fast when using in-memory column format tables

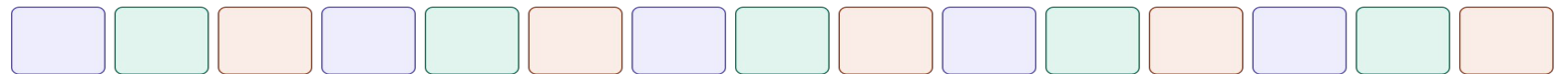
When to Use It?

Columnar format is best for:

- **Queries/aggregation** like SUM, AVERAGE, COUNT on specific columns (common in analytics/OLAP)
- Situations where you only need a **few columns** out of many

Day	A	B	C
Day 1			
Day 2			
Day 3			
Day 4			
Day 5			

Row-major (normal table storage)



To total flavour A, the CPU jumps over B and C, five separate times

Column-major (in-memory columnar)



All 5 values of flavour A sit side by side — one sweep gets the total

Enterprise Data-Store Server and Data Warehouse

Enterprise data, after data cleaning process, integrate with the server data at warehouse. Enterprise data server use data from several distributed sources which store data using various technologies.

All data merge using an integration tool. Integration enables collective viewing of the datasets at the data warehouse

Complex application-integration means the integration of heterogeneous application architectures and processes with the databases at the enterprise.

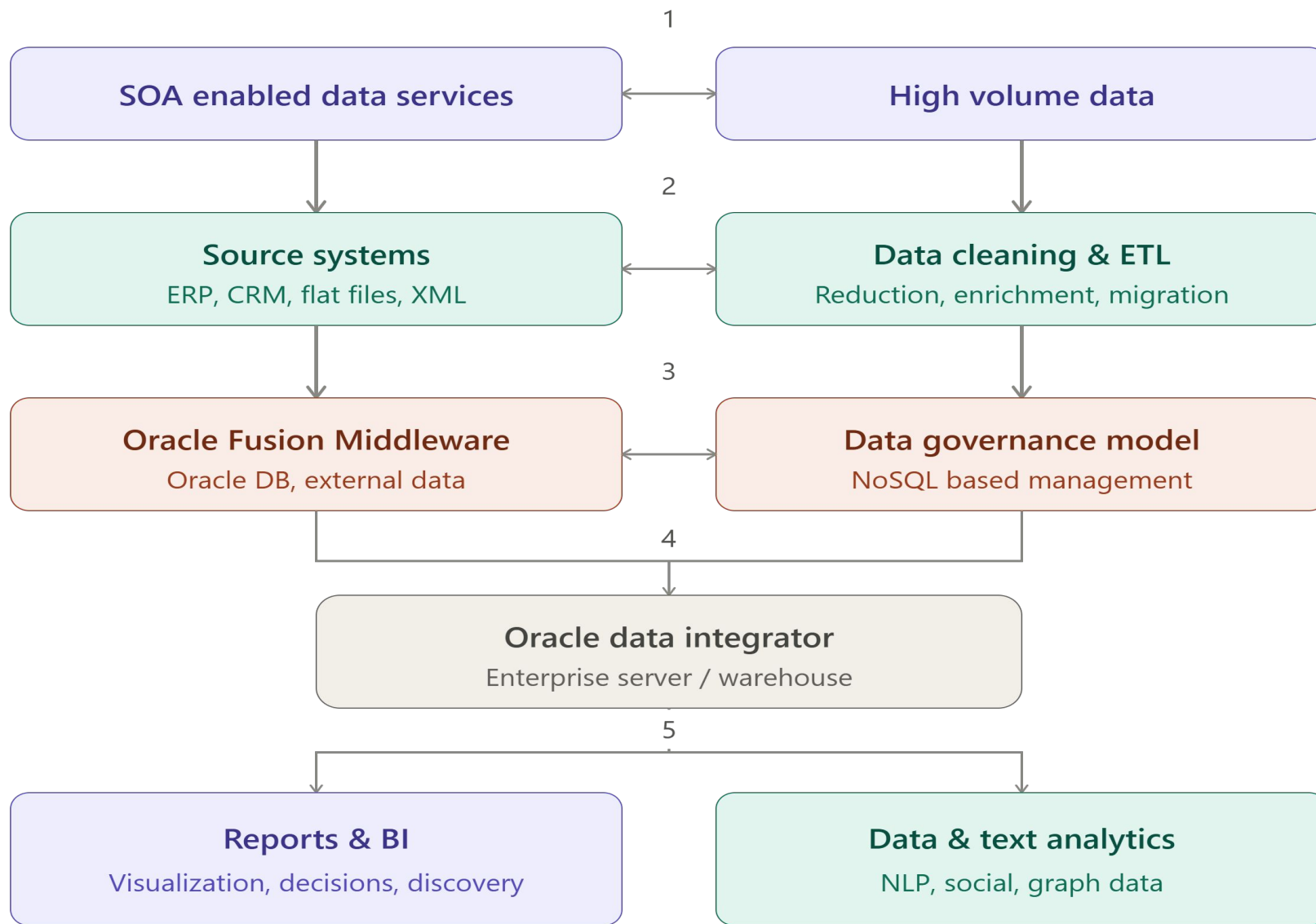


Figure 1.6 shows Steps 1 to 5 in enterprise data integration and management with Big Data for high performance computing using local and cloud resources for analytics, applications and services.

Big Data Storage: NoSQL

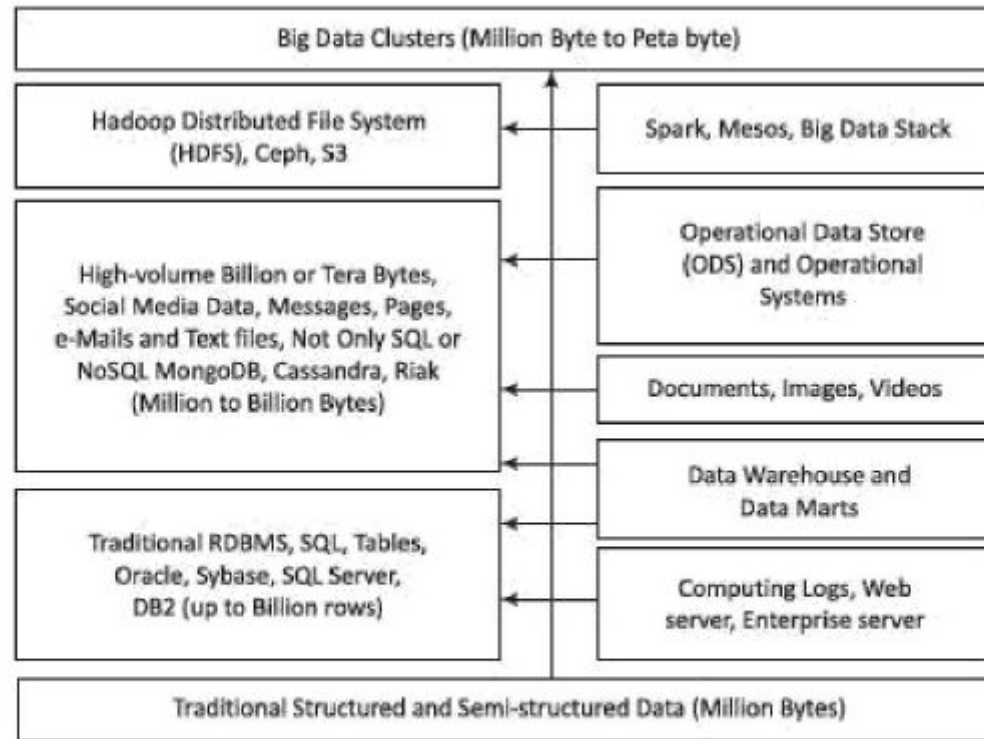
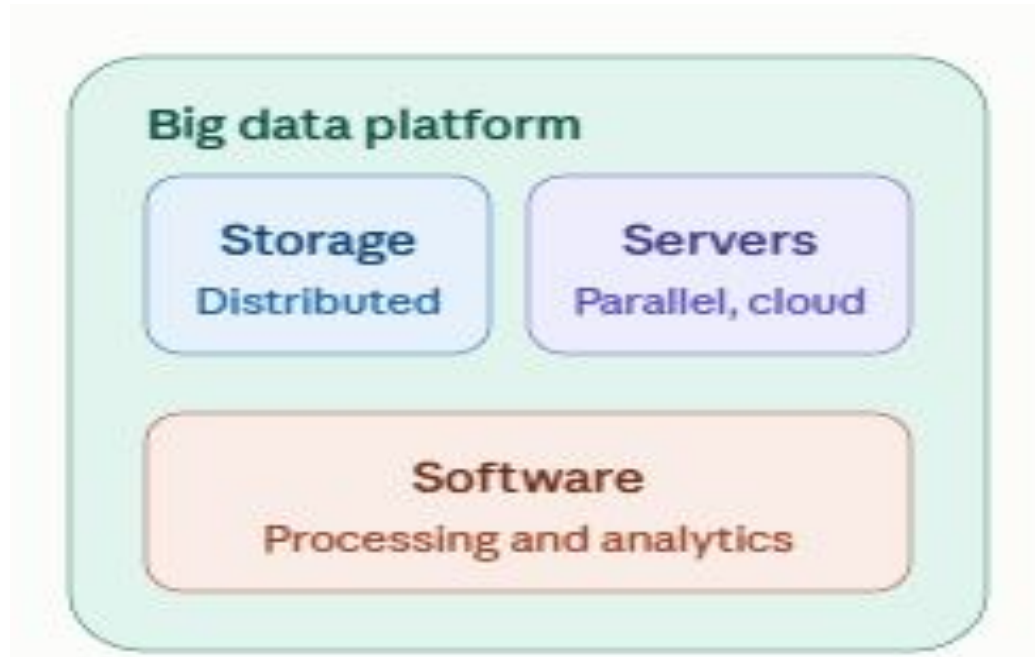


Figure 1.7 Coexistence of RDBMS for traditional server data, NoSQL and Hadoop, Spark and compatible Big Data Clusters

Big Data Platform

- A Big Data platform supports large datasets and volume of data.
- The data generate at a higher velocity, in more varieties or in higher veracity.
- Managing Big Data requires large resources of MPPs, cloud, parallel processing and specialized tools.



Bigdata platform should provision tools and services for:

- Storage, processing and analytics,
- Developing, deploying, operating and managing Big Data environment,
- Reducing the complexity of multiple data sources and integration of applications into one cohesive solution,
- Custom development, querying and integration with other systems, and
- The traditional as well as Big Data techniques.

Data management, storage and analytics of Big data captured at the companies and services require the following:

1. New innovative non-traditional methods of storage, processing and analytics
2. Distributed Data Stores
3. Creating scalable as well as elastic virtualized platform (cloud computing)
4. Huge volume of Data Stores
5. Massive parallelism
6. High speed networks
7. High performance processing, optimization and tuning
8. Data management model based on Not Only SQL or NoSQL
9. In-memory data column-formats transactions processing or dual in-memory data columns as well as row formats for OLAP and OLTP
10. Data retrieval, mining, reporting, visualization and analytics
11. Graph databases to enable analytics with social network messages, pages and data analytics
12. Machine learning or other approaches
13. Big data sources: Data storages, data warehouse, Oracle Big Data, MongoDB NoSQL, Cassandra NoSQL
14. Data sources: Sensors, Audit trail of Financial transactions data, external data such as Web, Social Media, weather data, health records data

Hadoop

Hadoop is an open-source software framework that is used for storing and processing large amounts of data in a distributed computing environment

- It is designed to handle big data and is based on **the MapReduce programming model**, which allows for the parallel processing of large datasets

A Big Data Platform has three core pieces working together **storage** (where data lives), **servers** (where processing happens), and **business intelligence software** (where insights get extracted).

This section focuses on the *storage* piece.

Mesos V 0.9

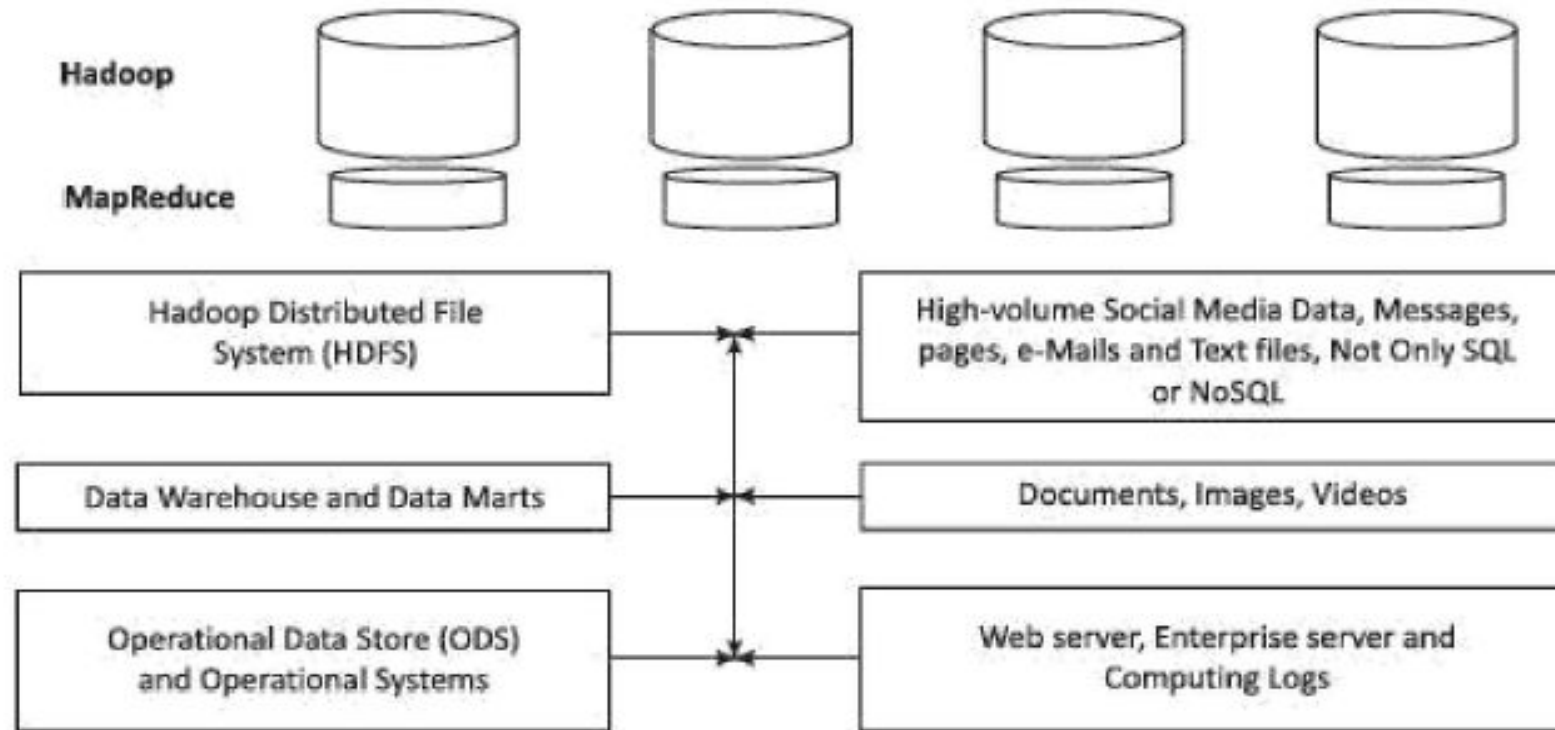


Figure 1.8 Hadoop based Big Data environment

Big Data Stack

"A stack consists of a set of software components and data store units."

It's a *combination* of multiple pieces working together: **software components** (processing engines, frameworks like Hadoop/Spark) plus **data store units** (the databases and file systems, like HDFS, MongoDB, Cassandra) that were described in Table 1.5

"Applications, machine-learning algorithms, analytics and visualization tools use Big Data Stack (BDS)"

Table 1.5 Tools for Big Data environment

Types	Examples
MapReduce	Hadoop, Apache Hive, Apache Pig, Cascading, Cascalog, mrjob (Python MapReduce library), Apache S4, MapR, Apple Acunu, Apache Flume, Apache Kafka
NoSQL Databases	MongoDB, Apache CouchDB, Apache Cassandra, Aerospike, Apache HBase, Hypertable
Processing	Spark, IBM BigSheets, PySpark, R, Yahoo! Pipes, Amazon Mechanical Turk, Datameer, Apache Solr/Lucene, ElasticSearch
Servers	Amazon EC2, S3, GoogleQuery, Google App Engine, AWS Elastic Beanstalk, Salesforce Heroku
Storage	Hadoop Distributed File System, Amazon S3, Mesos

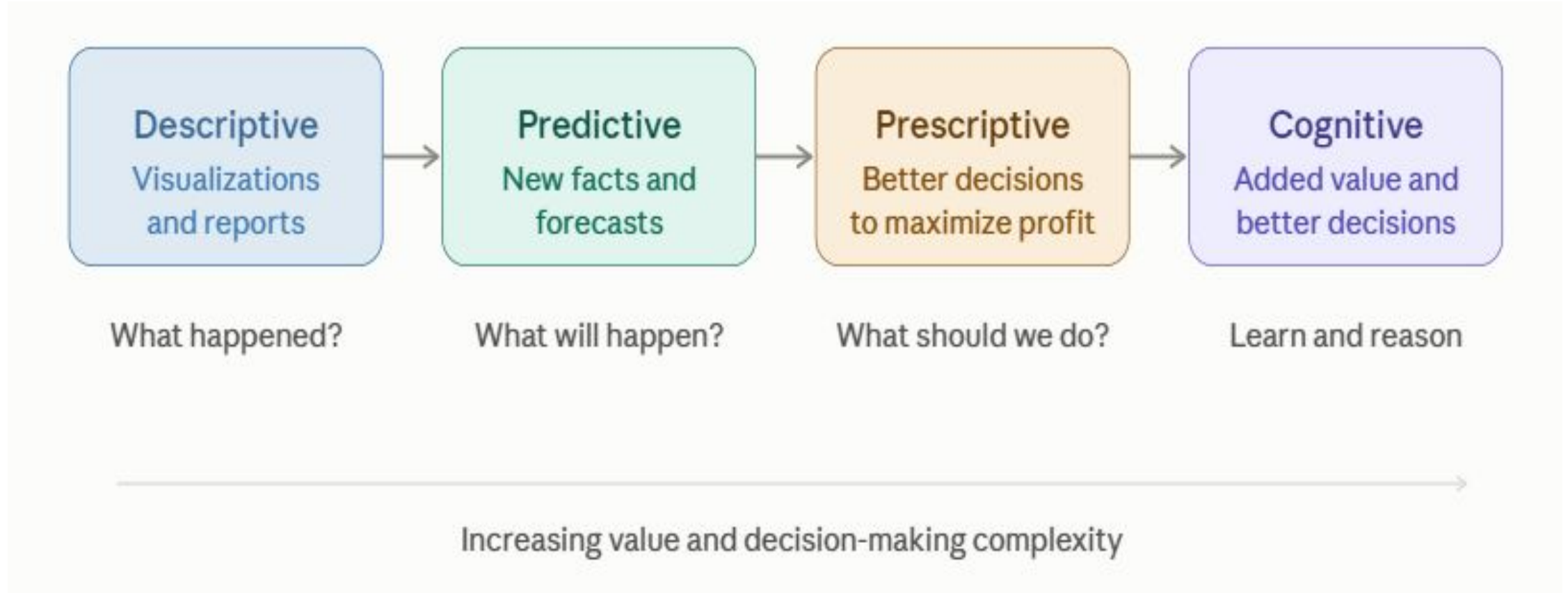
Big Data Analytics

- Data analysis need **pre-processing** of raw data and gives information useful for decision making.
- Analysis brings **order, structure and meaning to the collection of data.**
- Data is collected and analyzed **to answer questions, test the hypotheses or disprove theories.**

Data Analytics Definition:

"Analysis of a data is a process of inspecting, cleaning, transforming and modeling data with the goal of discovering useful information, suggesting conclusions and supporting decision making."

Phases in Analytics



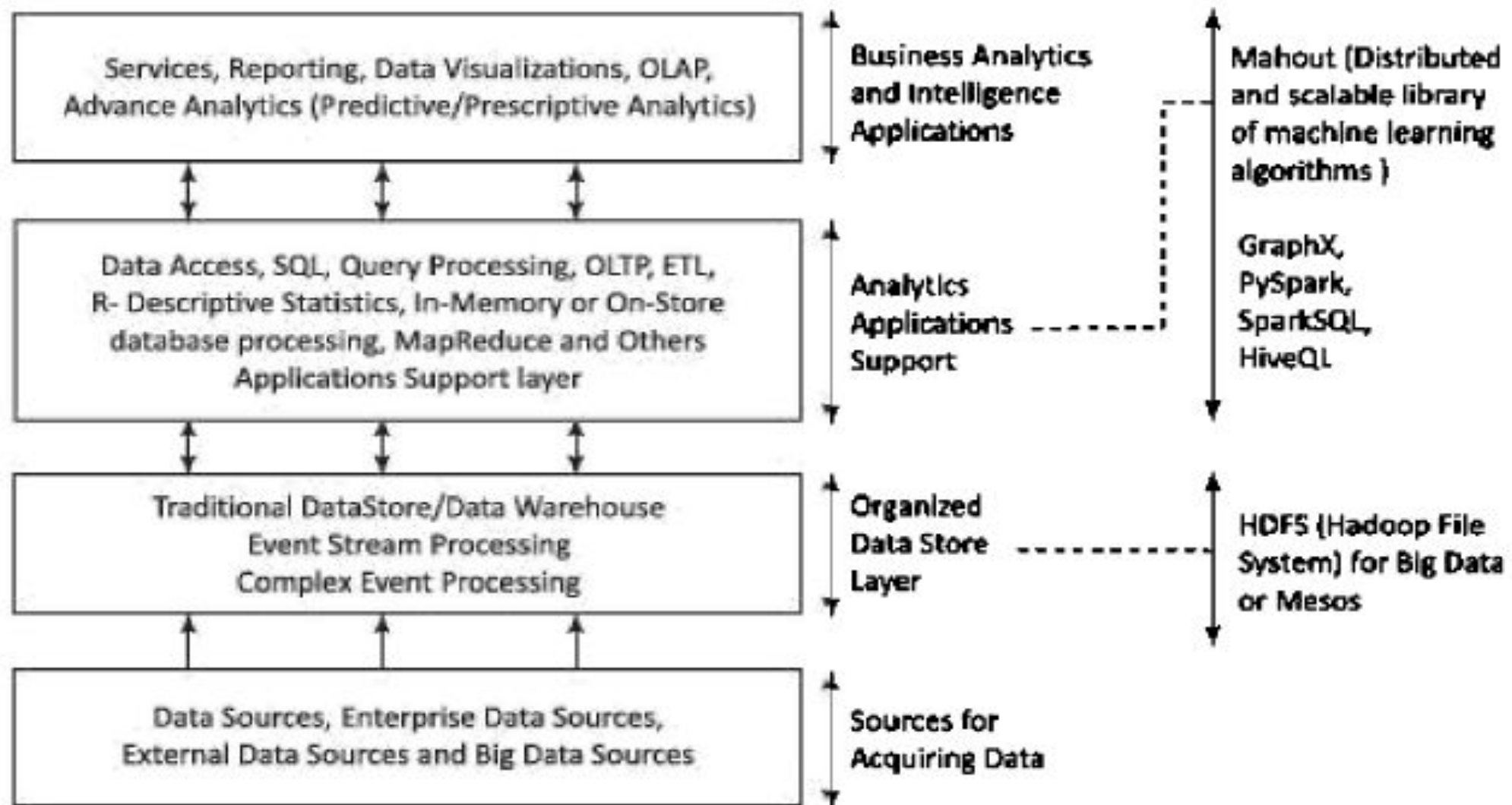


Figure 1.9 Traditional and Big Data analytics architecture reference model

Shows an overview of a reference model for analytics architecture. The figure also shows on the right-hand side the Big Data file systems, machine learning algorithms and query languages and usage of the Hadoop ecosystem.

Diagram contents (four horizontal layers, bottom to top, with a side box):

- **Layer 1 (bottom):** Data Sources, Enterprise Data Sources, External Data Sources and Big Data Sources
- **Layer 2:** Traditional Database/Data Warehouse, Event Stream Processing, Complex Event Processing
- **Layer 3:** Data Access, SQL, Query Processing, OLTP, ETL, R- Descriptive Statistics, In-Memory or On-Store database processing, MapReduce and Others Applications Support layer
- **Layer 4 (top):** Services, Reporting, Data Visualizations, OLAP, Advance Analytics (Predictive/Prescriptive Analytics) → Business Analytics and Intelligence Applications

- The captured or stored data require a well-proven strategy to calculate, plan or analyze.
- When **Big Data combine with high-powered data analysis**, enterprise achieve valued business-related tasks. Examples are:
 - Determine root causes of **defects, faults and failures** in minimum time.
 - **Deliver advertisements** on mobiles or web, based on customer's location and buying habits.
 - **Detect** offender before that affects the organization or society.

Berkeley Big Data Analytics Stack (BDAS)

*BDAS (Berkeley Data Analytics Stack) is an open-source Big Data software stack developed by the **AMPLab** at UC Berkeley*

The importance of Big Data lies in the fact that what one does with it rather than how big or large it is. Identify whether the gathered data is able to help in obtaining the following findings:

- 1) Cost reduction,
- 2) Time reduction
- 3) New product planning and development
- 4) Smart decision making using predictive analytics and
- 5) Knowledge discovery.

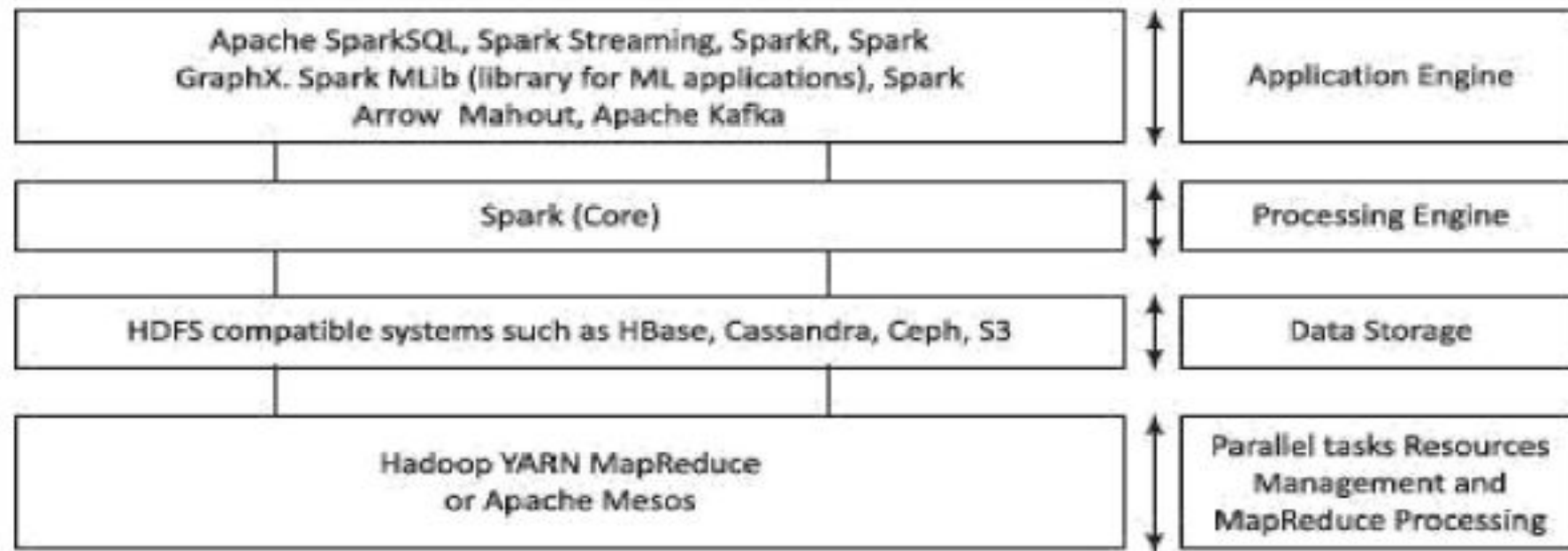


Figure 1.10 Four layers architecture for Big Data Stack consisting of Hadoop, MapReduce, Spark core and SparkSQL, Streaming, R, GraphX, MLib, Mahout, Arrow and Kafka

SLT: 1.7